CeTTa:
A Direct C Runtime for MeTTa with Verified Translation

Zar Goertzel
Oruži

July 23, 2026

Abstract

MeTTa is a reflective, non-deterministic meta-language with three actively maintained runtimes: Hyperon Experimental (HE, Rust), PeTTa (Prolog), and CeTTa (C). This paper describes CeTTa, a direct C implementation of the HE MeTTa evaluator that passes a 535-test MeTTa regression suite (a 539-row manifest): 405 passed / 0 failed on the default build and 401 passed / 0 failed on the MORK-bridge build, plus a heavy-golden lane at 17 passed / 0 failed (suite snapshot 2026-07-02, commit 9237afd), backed by a 23-case HE I/O semantic conformance gate against upstream HE v0.2.10. On cross-runtime benchmarks—genomic-inference rows re-receipted 2026-07-02 on commit 9237afd, branching-search rows measured 2026-06-27—CeTTa is fastest on startup-dominated shallow inference and on several genomic-inference rows (the 183-pair PLN evidence core, the full 1,399,997-line WM-PLN raw-loader checkpoint, and the 1,411,430-hypothesis deep genomic backward-chaining row), while PeTTa wins on pure branching search (the 85,013-proof Metamath depth-5 benchmark) and on direct 8-Queens search. A rerun-backed cross-engine chaining suite built on community chainers (Geisweiller's dependent-typed chainers and Metamath proofs; Treutlein's PeTTaChainer problems) shows that CeTTa's shallow advantage is startup-dominated rather than a general scaling property. Historical rows whose current drivers now disagree or fail—the old cross-engine miner speed table, the old full-1.4M WM-PLN scaling headline, and BTree—are explicitly excluded from the revised benchmark ladder. The earlier tilepuzzle regression was root-caused to unbounded growth of the evaluation arena and closed by a tail-safe evaluation-arena garbage collector (on by default), restoring tilepuzzle as benchmark evidence: the full 181,440-state 8-puzzle search now completes in 8.80 s at 233 MiB peak resident memory (commit 9237afd). The runtime is organized as an explicit profile family: `he-compat` mirrors Rust HE literally (including its quirks); the principled `he` line carries exact rationals, true division, and per-case recorded divergences; `he-extended` adds labeled extensions; `he-prime` adds dependent telescopes. The one-step reduction relation is reflected into MeTTa itself as a modal surface (`transitions`, `box`, `diamond`, normal-form tests) aligned with the OSLF LanguageDef → ReductionSpan → $\Diamond/\Box$ pipeline proven in Lean, and the first rule classes (grounded dispatch, argument congruence, equation matching) are welded to a reflectable small-step rule table whose rule-removal tests prove the engine genuinely depends on the declared rules. Two 2026-07 developments extend the architecture: parsers *generated from the authored grammars themselves* (per-language LanguageDefs compiled to native readers with differential, mutation, and purity gates — the pipeline caught two defects in the authored grammars before execution), and `-lang prime`, a *distinct* lazy sibling language (branch-local call-by-need with call-time choice, opaque Need-cell capabilities, occurrence-bag results, and a policy-neutral certified search-control layer), whose living specification and design principles are distilled in the appendices. A correctness-gated four-engine benchmark study (adding JeTTa and MeTTaScript) locates each runtime's competence zone and measures the per-call and sharing costs that motivate the abstract-machine roadmap, whose derivation methodology now carries kernel-checked Lean foundations. The runtime features SymbolId-based dispatch (38% instruction reduction), structured space kinds (queue, hash, stack), a pluggable match backend with PathMap/MORK integration, and compiled space artifacts via MORK's `.act` format for zero-copy indexed loading. It is backed by a sorry-free Lean proof (5,400+ lines) establishing semantic correspondence between HE and PeTTa evaluation on the pure fragment, including import commutativity and operational bridges for state and space allocation. A kernel-checked HE′ telescope formalization extends the type system with dependent binder semantics. The RhoCalc reaction machine's eval-at-COMM semantics is given a witness-free, axiom-clean Lean characterization: the quiescent run of the canonical payload-evaluation process equals its structurally observable outcome set. We present a benchmark ladder from shallow evaluation through large-scale genomic inference and a cross-engine chaining suite, a freshness-first performance re-audit and triage, a specification gap analysis, and a three-machine architecture (local search, indexed shared-space, type elaboration) informed by GPT-5.4 Pro architectural review.

1 Introduction

MeTTa [1] is a reflective, typed, non-deterministic language designed as the core language of the OpenCog Hyperon AGI framework. Three implementations exist:

HE Hyperon Experimental (Rust, v0.2.10). The reference implementation with Python bindings and a large standard library.

PeTTa Prolog-based MeTTa (SWI-Prolog). Leverages Prolog's SLD resolution for backward chaining and pattern matching.

CeTTa Direct C evaluator (this work). Targets the HE spec with arena allocation, discrimination-trie indexing, and tail-call optimization.

Contributions.

1. A direct C runtime passing a 535-test MeTTa regression suite (539-row manifest; default build 405/0, MORK-bridge build 401/0, heavy-golden lane 17/0) plus 112 profile tests, validated against upstream HE v0.2.10 as oracle.
2. An explicit profile family (`he-compat` / `he` / `he-extended` / `he-prime`) with per-case recorded divergences and a fail-by-default conformance harness: the current per-build gate replays 23 HE I/O semantic fixtures against the HE v0.2.10 baseline, while broader generated catalogs remain audit material rather than headline claims (Section 3).
3. A reflective modal LTS surface over the evaluator's one-step reduction relation—successor set, $\Box/\Diamond$, normal-form tests—with runtime De Morgan self-tests mirroring the Galois connection proven in the Lean OSLF framework.
4. The first specification weld: a reflectable small-step rule table gating grounded dispatch, argument congruence, and equation matching, with anti-decorative rule-removal tests. This converts a cross-branch one-step soundness bug (missing argument congruence) into a structurally prevented class.
5. A bidirectional HE$\leftrightarrow$PeTTa translator (Prolog + executable Lean), with a sorry-free Lean proof of semantic correspondence.
6. Cross-runtime benchmarks (genomic-inference rows re-receipted 2026-07-02 on commit 9237afd; branching-search rows measured 2026-06-27, pre-GC): CeTTa is 10–100× faster

than HE on shallow evaluation, wins several current genomic-inference rows (including the full TFBS-inclusive WM-PLN raw-loader checkpoint), while PeTTa is 4.8× faster on the audited depth-5 Metamath proof-search row and faster on direct 8-Queens search; only still-unresolved rows are quarantined from the revised ladder.

7. A tail-safe evaluation-arena garbage collector, on by default, differentially validated (not verified) against the collector-off reference across the fast regression corpus (406/406 agree) and under ASan/UBSan with provenance assertions (404/404 agree); its headline effect is the restored tilepuzzle capacity row—the full 181,440-state search completes in 8.80 s at 233 MiB, versus unbounded arena growth before (Section 7.2).

8. A characterization of the semantic boundary between HE and PeTTa: the shared fragment (85% of programs) runs identically; the divergence is in free-variable scoping across non-deterministic branches.

9. A specification gap analysis of non-determinism plus mutable state, with 8 concrete examples showing that sequential-per-iteration semantics (CeTTa's default) is correct in 6/8 cases, while HE's interleaved side-effect evaluation loses atoms in real code (the pattern miner).

10. `with-space-snapshot`: a CeTTa extension for branch-isolated mutation, enabling semi-naive forward chaining and speculative search without the side-effect hazards of shared mutable state.

11. A witness-free, axiom-clean Lean characterization of the RhoCalc reaction machine's eval-at-COMM semantics: the quiescent run of the canonical payload-evaluation process equals its structural-congruence-observed outcome set, with no certification witness in the carrier (Section 5.4).

2 CeTTa Architecture

CeTTa is a direct evaluator, not a compiler or boot pipeline. It implements the six mutual functions of the HE evaluation spec (`EvalAtom`, `MettaCall`, `InterpretExpression`, `InterpretFunction`, `InterpretArgs`, `InterpretTuple`) as C functions with a shared arena allocator.

2.1 Core Design

Arena allocation. All atoms are allocated from 64 KB arena blocks. No per-atom `malloc`/`free`. Batch deallocation at scope exit.

Symbol interning. A hash table maps symbol strings to unique pointers, enabling O(1) pointer comparison for symbol equality.

Equation index. Equations `(= lhs rhs)` are indexed by head-symbol hash into 256 buckets, with a discrimination trie within each bucket for fast LHS matching.

Substitution tree. An epoch-based discrimination tree (`SubstTree`) eliminates per-candidate variable renaming during equation queries by tagging indexed variables with monotonic epochs.

Tail-call optimization. A `TAIL_REENTER` trampoline with `goto tail_call` eliminates stack growth for recursive equation dispatch.

Unlimited fuel. Default fuel is -1 (unlimited), matching the HE spec. Stack overflow is caught via `sigaltstack` + `SIGSEGV` handler with a clean error message.

Two-stage stdlib. The standard library is precompiled into a binary blob at build time (`cetta-stage0` → `-compile-stdlib` → blob → `cetta`), giving 6 ms startup.

2.2 Test Suite

CeTTa is validated against the HE CLI (v0.2.10) as oracle. Golden regression files are oracle-validated when introduced; `make test` replays them on every build. As of the 2026-07-02 snapshot on commit 9237afd: **405 passed, 0 failed, 71 skipped** on the default build and **401 passed, 0 failed, 75 skipped** on the MORK-bridge build (all bridge sub-lanes green), plus 112 profile tests covering module imports, foreign (Python) function dispatch, numeric-tower and profile-surface splits, and extension surfaces. Two further lanes are fail-by-default: `make test-step-rules` disables individual rule-table entries and *requires* the witnesses to fail (Section 3), and the semantic conformance suite re-runs 23 curated HE I/O fixtures live against upstream HE (Section 3.2).

3 Specification-Driven Semantics: Profiles, Conformance, and the Small-Step Rule Table

CeTTa is organized around an explicit principle: *the semantics is a named object, not an emergent property of the implementation.* This section describes the profile family, the oracle-conformance campaign against upstream HE, the evaluator bug classes that campaign exposed, the reflective modal surface over the one-step reduction relation, and the first welded rule pack that makes the reduction rules themselves runtime data.

3.1 The Profile Family

Profiles are distinct semantic products, implemented as additive layers—never as visibility masks over one shared universe:

`he-compat` A literal mirror of Rust HE v0.2.10, including its quirks: integer division for `/`, the strict three-argument `if` surface, Rust's library inventory behavior. Compatibility means matching upstream *behavior*, not reproducing its library set; library availability is out of scope for conformance.

`he` The principled line. Spec-faithful where the written MeTTa specification speaks; where HE's implementation and its specification diverge, the divergence is adjudicated and recorded per-case. Carries exact arbitrary-precision rationals, true division for `/`, floor division `//`, Euclidean `math:mod`, and the clean `math:` namespace over the Rust-heritage `*-math` ops. Extensions that accept more programs (e.g. two-argument `if`) are deliberate, recorded surface choices.

`he-extended` `he` plus labeled CeTTa extensions: typed space kinds, `with-space-snapshot`, the `lts` reflection surface, MORK/PathMap-backed spaces.

`he-prime` The dependent-telescope extension (Section 10).

Two rules keep this family honest. First, unknown symbols stay inert: MeTTa's philosophy is to leave uninterpreted what it does not know how to interpret, so no profile ever raises an "unsupported feature" error—a profile determines what is *interpreted*, never what is *legal*. Second, profile differences are explicit splits with named tests (e.g. `profile_if_compat_arity` pins that `he-compat` rejects two-argument `if` exactly as upstream does, while `he/he-extended` accept it as a recorded extension).

3.2 Oracle Conformance

For `he-compat`, upstream HE v0.2.10 is the sole oracle, and the harness is a fail-by-default gate, not a green-manufacturing report. Comparison is by *result-bag equality*—multisets of parsed results, never byte-equality of output text—so formatting differences cannot masquerade as semantic agreement, and duplicate results (which are semantically real in MeTTa) are counted.

Current HE I/O semantic gate. The per-build oracle gate (`make test-he-compat-semantic-suite`) replays 23 curated HE I/O fixtures against the upstream HE v0.2.10 baseline. The July 2 receipt reports 23/23 CeTTa passes, 0 failures, and a passing baseline check. Six of those fixtures are core anchors into the Lean-side HE I/O conformance file. This is the current conformance claim used by the paper.

Archived catalog material. Earlier generated rule-capacity and broad-corpus catalogs remain useful triage artifacts, but they are not treated as current paper evidence unless their driver emits the claimed counts on the audited tree. In particular, this revision does not carry a 24-capacity maturity table or a broad-corpus count as a live conformance result.

Adjudicated divergences. Where CeTTa deliberately diverges from upstream, each case carries independent grounds. Three examples: upstream's post-0.2.10 `function`/`NoReturn` behavior contradicts its own written specification (a deliberate loop fix that left the document stale); upstream's `get-doc` errors on HE's own documentation example (an arity regression); and the user-visible result of top-level `eval` on a non-reducible term is genuinely under-determined by the specification—both engines deviate from the literal instruction-layer reading, in opposite directions.

The conformance material carries an explicit `no_runtime_trace_or_certificate_surface` flag, enforced by the suite: conformance is *testing*, and the specification is what gets *proven about* in Lean. CeTTa does not emit execution traces or certificates, and a per-run check is never presented as a proof.

3.3 Evaluator Bug Classes the Campaign Exposed

Three bug classes were found and fixed, all instructive.

The Empty algebra. When overlapping equations yield a branch that reduces to `Empty`, two dual defects appeared: the surviving sibling branch was returned without being re-driven to applicative normal form (violating the spec's applicative order), and in other shapes `Empty` itself leaked into the visible result bag (the spec is explicit that `Empty` "is not returned among other results"). The fix treats the evaluator's outcome set as an algebra with `Empty` as its absorbing zero and re-drives surviving branches independently of their siblings. In modal terms (Section 3.4): the successor algebra must satisfy $\Diamond\bot = \bot$.

The one-step congruence bug. The reflective one-step surface reported `(+ 1 (+ 2 3))` as a *normal form*: the one-step relation lacked argument congruence (stepping a reducible argument when the head cannot fire), while the big-step evaluator had it. Because the one-step relation and the evaluator were separately hand-maintained C branch sets, the gap replicated identically across three development branches and was invisible to every test that only exercised head-reducible terms. The bug was diagnosed through the modal surface itself ($\Box\bot$ holding on a reducible term), fixed as deterministic left-to-right congruence matching the evaluator's applicative order—and is the direct motivation for the rule-pack weld in Section 3.5.

The switch/switch-minimal conflation. Upstream HE types `switch` as `(-> %Undefined% Expression %Undefined%)`—the scrutinee is *evaluated* before matching—while `switch-minimal` is `(-> Atom Expression Atom)`, matching the raw scrutinee structurally. CeTTa routed both spellings through one structural handler, so `(switch (+ 1 2) ((3 three) (($x $y) (got $x $y))))` returned `(got 1 2)` where upstream returns `three`. The defect was exposed not by testing but by the certification campaign (Section 3.6): modeling the rules in Lean forced a precise reading of the two upstream type contracts, and an oracle probe confirmed the divergence. The fix splits the rule into `HES_Switch` (evaluated scrutinee) and `HES_SwitchMinimal` (structural), with a distinguishing witness pair pinned in the test suite.

3.4 The Reflective Modal Surface

The library surface `lts` (generic, step-parameterized) and `lts:he` (bound to the HE evaluator's one-step relation) reflect the one-step reduction relation into MeTTa as data:

- `lts:he:transitions` emits the one-step successor set of an atom as quoted states (zero results on quiescent atoms); `lts:he:trace-2` enumerates two-step traces.
- `lts:he:is-normal-form` is exactly $\Box\bot$ (every successor satisfies falsity, i.e. no successor exists); `lts:he:is-can-step` is $\Diamond\top$.
- `lts:he:box` ϕ and `lts:he:diamond` ϕ generalize these to arbitrary unary predicates: $\Box\phi$ holds when every one-step successor satisfies ϕ (vacuously at normal forms), $\Diamond\phi$ when some successor does.

The De Morgan law $\Box\phi = \neg\Diamond\neg\phi$ is installed as a runtime self-test; it is the executable mirror of the Galois connection proven once and for all in the Mettapedia OSLF framework, where LanguageDef $\to$ RewriteSystem $\to$ ReductionSpan $\to$ $(\Diamond, \Box)$ is derived generically and `langGalois` establishes the adjunction. Native behavioral types in the sense of Native Type Theory arise as predicates over exactly this reduction span; the runtime surface and the Lean derivation describe the same object at two levels.

Multiplicity discipline: `transitions` preserves the result *bag* (duplicate successors are semantically real), while the modal predicates quotient to reachability. Evaluation and conformance compare bags; $\Diamond/\Box$ use support. Result sets are never deduplicated for speed.

3.5 The Small-Step Rule Table

The congruence bug demonstrated the structural disease: reduction rules as anonymous, duplicated C branches are invisible, unremovable, and untestable-by-absence. The cure is to make the rule set a *named, reflectable object* the evaluator provably depends on.

Three granularity strata are kept distinct. *HEFine* is the Lean `mettaHE` instruction machine (59 rules over interpreter states; the proof reference). *HE small-step* is the coarse user-visible one-step relation—one observable surface rewrite, e.g. `(+ 1 (+ 2 3))` $\to$ `(+ 1 5)`—which is what the `lts` surface exposes. *HEBigStep* is full evaluation. The table encodes the small-step relation; its relationship to the fine machine is a collapse/simulation with explicit provenance links (each coarse rule names the fine rules that justify it), never a label identity.

The runtime object, `CettaLangDefPack`, carries: language, profile, and granularity identifiers; a schema version; a source digest over the canonical rule-descriptor text; the rule table; a disabled-rule mask populated from the `CETTA_LANGDEF_DISABLED_RULES` environment variable; and a *per-rule claim level* recording the strongest evidence behind each rule (Section 3.6). Ten rule classes are live and gate the *only* implementations of their behavior in the one-step subsystem: grounded dispatch, leftmost expression congruence, equation match, `eval`, `chain`, `let`, `let*`, `case`, `switch`, and `switch-minimal` (the last two split per their distinct upstream contracts, Section 3.3). Quote/return and empty/error quiescence are structural rows: accounted in the table with written reasons, but not gateable, since disabling them would make the semantics incoherent rather than smaller. Each row carries a provenance string naming the fine-machine rules that justify it. The pack is reflected into MeTTa by `lts:he:step-rules`.

The load-bearing test is *anti-decorative*: a dedicated lane re-runs the witness suite with each live rule disabled and *requires* the witnesses to fail (the successor set must go empty; normal-form must flip)—baseline plus ten rule-removal failures confirmed. If the witnesses still passed with a rule removed, a duplicate hidden code path would exist—precisely the disease that produced the congruence bug. A disabled special form goes *inert* (its head stays uninterpreted; congruence still applies), per the principle that unknown symbols are data, never errors. The lane fails closed in both directions and runs on both the default and MORK-bridge builds.

Claim levels are deliberately *excluded* from the digest: the digest fingerprints rule *content* (names, profiles, provenance, schema), while claims are *evidence about* that content. Upgrading a rule's certification therefore must not—and verifiably does not—move the semantic object: the digest (`fnv1a64:12eec5b655a75904` at the time of writing) stayed fixed across every claim flip. The pack is, by design, a hand-authored object kept honest by the removal lane and the digest; we deliberately parked an earlier plan to generate specialized C from the rule source, because the weld's value lies in the named semantic object and its fail-closed tests, and code generation is machinery the certification program does not need.

3.6 Lean Certification of the Rule Table

Each per-rule claim takes one of three values, in increasing strength: *bag-tested-adequate* (the rule's behavior matches the legacy path and oracle bags on the witness corpus), *rule-sound-on-fragment* (a compiled, sorry-free Lean theorem proves the rule absorbed by the official declarative semantics on an explicitly characterized fragment), and *rule-sound* (the same, unconditionally). A claim flips only when the compiled theorem exists, so the reflective surface can never overclaim. The certification route is *absorption*: for each coarse rule, a theorem that whenever the rule relates $a \to a'$, the official declarative judgment of the Mettapedia HE formalization (`MettaCall`, `MinimalStep`, or `EvalAtom`) already relates the same atoms. At the time of writing five of the ten live rules carry certification:

rule	claim	Lean witness
`HES_GroundedDispatch`	rule-sound	`mettaCall_absorbs_grounded_dispatch`
`HES_EquationMatch`	rule-sound	`mettaCall_absorbs_equation_match`
`HES_Eval`	rule-sound	`minimalStep_absorbs_eval`
`HES_Chain`	rule-sound-on-fragment	`minimalStep_absorbs_chain_source/_subst`
`HES_LeftmostExprCongruence`	rule-sound-on-fragment	`evalAtom_absorbs_congruence_frag`
`HES_Let`, `LetStar`, `Case`, `Switch`, `SwitchMinimal`	bag-tested-adequate	in progress

The certified fragment for the congruence and chain rules is characterized positively: untyped constructor-headed expressions with quiescent spectator arguments and a grounded-dispatch or equation-match redex. Its supporting theory is a short ladder of compiled lemmas: a quiescence domain on which atoms officially self-evaluate, uniqueness of that self-evaluation, transport of official evaluation across a single changed argument, and absorption of inner fragment steps—about which the honest surprises accumulated (e.g. a bare `Error` *symbol* at tuple head becomes error-shaped, so the fragment must exclude it explicitly).

The five remaining rules are upstream *stdlib sugar*: their one-step behavior is the upstream definition of `let`, `let*`, `case`, `switch`, and `switch-minimal` as `unify/chain` programs. Certifying them required a *matcher bridge*: relating the public equation-query surface to the official `matchAtoms/mergeBindings` semantics. The key insight from this development stage was that the real seam was *not* merely a direct one-way-to-two-way matcher comparison. The public HE query surface had historically routed equation lookup through the simplified `simpleMatch` path, while the faithful two-sided matcher already existed separately. We therefore split the bridge into three explicit layers.

First, `DeclMatchSpec.lean` states the author's `match_atoms` semantics declaratively and proves the executable `matchAtoms` helper sound and complete against it (`matchAtoms_sound`, `matchAtoms_complete`). Second, `DeclMergeSpec.lean` does the same for `merge_bindings/add_var_binding/add` proving `mergeBindings_sound` and `mergeBindings_complete`. Third, `QuerySurfaceAgreement.lean` proves a bounded adequacy theorem for the *public* query interface: `queryEquations_ground_two_way_adequate` and its `queryEquationsAgainstVisible`-companion show that, on the ground-query fragment and under the explicit freshening adequacy hypothesis `EquationFresheningAdequate`, the repaired `matchAtoms/mergeBindings` surface and the legacy `simpleMatch`-based scan agree in both directions up to replay fuel.

The earlier singleton-seed factorization theorem remains one ingredient inside this story, but it is no longer the whole story. The decisive insight is the G3/G3b split: equality-threading becomes observable only when the queried atom itself still contains variables, because that is the point at which `Bindings.resolve/Bindings.apply` must honor equalities rather than plain assignments. We therefore bank the ground-query adequacy result as the honest current certification surface, and carry the variable-bearing query case as an explicit next obligation rather than silently pretending the legacy surface was already faithful.

4 Verified HE↔PeTTa Translation

4.1 Bidirectional Translator

A bidirectional translator converts between HE and PeTTa dialects. It is implemented both as relational Prolog (`he_petta_relational.pl`, with explicit freshness threading) and as computable Lean functions (`translateHE`, `translatePeTTa`).

The translator handles 8 HE→PeTTa rewrite rules (`chain`→`let`, `collapse-bind`→`collapse`, `nop`→`let $fresh X ()`, etc.) and 5 PeTTa→HE rules (`progn`→nested `let`, `prog1`→`let` with result capture, etc.). All 437 real `.metta` files parse and translate successfully.

4.2 Sorry-Free Lean Proof

The translation is backed by a **sorry-free, axiom-free** Lean proof across two files totaling 4,979 lines and 102 definitions/theorems:

`HEPeTTaSound.lean` (1,063 lines, 44 theorems.) Proves that HE's equation matching corresponds to PeTTa's `ruleApp` evaluation. The key result:

> `simpleMatch_applyBindings_comm`: if HE's `simpleMatch` succeeds on two `PureTranslatable` atoms, then applying the translated bindings via `heBindingsToOSLF` to the translated pattern recovers the translated target.

This commutation property, combined with the existing `matchPattern_applyBindings_complete` from the OSLF framework, establishes that a PeTTa `matchPattern` witness exists whenever HE's `simpleMatch` succeeds.

`HEPeTTaTranslate.lean` (3,916 lines, 58 theorems.) The executable translator with:

- `translateHE_translatable`: translation preserves `Translatable` (output admits `atomToPattern`).
- `translateHE_preserves_pureTranslatable`: translation preserves the stronger `PureTranslatable` fragment.
- `translatePeTTa_translatable`: same for the reverse direction.
- 9 `#eval` tests matching Prolog output on concrete atoms.
- 6 `rfl` examples kernel-verified by Lean.
- Roundtrip idempotence examples (kernel-verified).

5 Reflective ρ-calculus and eval-at-comm

Under `-lang rhocalc` the same runtime stops being a term rewriter and becomes a concurrent process engine: its unit of work is not a rewrite but a *reaction*, a rendezvous between a sender and a receiver on a shared channel. This brings channels with private names and genuine concurrency; and because the calculus is *reflective*, a received name can be *run*, so a communication can itself be a computation. The MeTTa library that exposes that last capability is *rhometta*. We build up to it from the pure calculus.

5.1 The pure calculus

The calculus is Meredith and Radestock's reflective higher-order ρ-calculus [4]. Its one reduction rule, COMM, is a handshake (Figure 1): a sender `x!(m)` on channel `x` beside a receiver `for(y <- x){P}` on that same channel react; the message is consumed and the receiver resumes with `m` substituted for its bound name `y`:

$$x!(m) \mid \mathsf{for}(y \leftarrow x)\{P\} \xrightarrow{\text{COMM}} P[y := m].$$

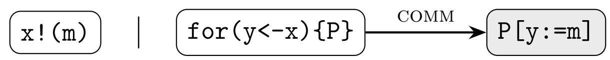


Figure 1: A reaction: a send and a matching receive on channel `x` rendezvous; the receiver continues with the message substituted in.

The rest of the vocabulary is small. `0` is the inert process; `P | Q` runs `P` and `Q` concurrently. Two reflection operators connect processes and names: `@P` *quotes* a process into a name, and `*x` *drops* (runs) the process a name quotes, with `*@P = P`. Channels are themselves quoted processes — this is what makes the calculus higher-order. CeTTa implements exactly these six strict-core constructors — `rho:nil`, `rho:par`, `rho:send`, `rho:recv`, `rho:quote`, `rho:drop` — with COMM as the only reduction. A program is reduced to quiescence by `cetta -lang rhocalc` *`file`*, written either in a MeTTa-style s-expression surface (`.mrho`) or a Rholang-like surface (`.rho`); the runtime keeps a live index of sends and receive-sites, rebuilt after each reaction. The deterministic (canonical) scheduler selects the key-least successor by streaming candidate keys against the best bound found so far instead of materializing and sorting the whole frontier, which keeps single-step selection cheap on wide parallel terms.

5.2 Reflection is enough

A surprising amount follows from COMM and quote/drop alone, with no further primitives. Each item below is a standalone `-lang rhocalc` program.

- *Name mobility.* A receive that drops its argument, placed beside a send carrying a quoted send, reduces to that inner send made live: a name has travelled across a channel and been activated.
- *Fresh names.* Quoting a distinct process yields a distinct name, so a receiver can mint and use a name it never mentioned syntactically.
- *Replicated input.* An input-guarded server can re-create its own listener: after serving a message it leaves a copy of the receiver in the residual, so the channel keeps serving. Persistent receive is *derived*, not built in.
- *Recursion and divergence.* The same self-recreation, left ungated, runs without bound (the engine reports its reduction limit); recursion needs no dedicated operator.

Private channels and information hiding come along for free: a payload exchanged on a name no other process holds is unobservable apart from whatever completion signal the protocol chooses to publish.

The capstone is *eval-at-COMM*. Let the receiver do nothing but drop what it receives:

$$\texttt{eval\_drop}(x,p) \;\equiv\; \underbrace{x!(@p)}_{\text{send the quoted program } p} \;\big|\; \underbrace{\mathsf{for}(y \leftarrow x)\{\, *y \,\}}_{\text{run whatever arrives}} .$$

The COMM delivers the name `@p`; the drop `*@p` runs `p`; the process settles at the result of running `p`. A handshake has become an evaluation step, and since `p` may itself evaluate to more process, a reaction can plant the next reactions.

5.3 Profiles

A *profile* names a variant of the reduction relation together with the grammar it accepts. `strict-core` is the pure asynchronous spine above: it admits only the six core constructors, and a non-core form is *rejected* rather than left inert. The `cost` profile accepts a richer resource-annotated dialect in which every reaction must be *funded* from purses located on its channel, and every run emits a causal receipt of its spending (Section 5.7).

5.4 Certified reduction

Does eval-at-COMM report the result of running the payload, or could the machinery quietly change the answer? We settle this in Lean. Run `eval_drop(x,p)` to quiescence and collect every outcome observable *up to structural rearrangement* of the process; that set is exactly the results a correct evaluation of `p` would produce, and the statement refers only to what is observable, never to an internal "this is the answer" tag. The theorem `rhometta_bridge_scObserved` (in `RhometaReduction.lean`) equates the concrete quiescent run of `eval_drop(x,p)` with `RhometaSCObservedOutcomes` of the same process — the outcomes visible up to structural congruence — with no result-certificate carried in the answer. Two ingredients are load-bearing. First, a complete canonical-form normalizer `nfAtom`: two processes are structurally congruent exactly when their normal forms coincide, which unblocks inductions that otherwise stall on the symmetry and transitivity of congruence. Second, one hygiene side condition `noBoundUnderQuote`: the communicated variable must not occur under a quote, since a payload that hides a bound occurrence inside a quoted name defeats the characterization (a concrete counterexample exhibits exactly that failure). The development carries no `sorry`. A separate caveat on the trusted base: its conformance and DSL fixtures use `native_decide`; the bridge theorem's own module does not.

5.5 rhometta: MeTTa at the reaction

eval-at-COMM becomes a way to program once the payload is a MeTTa term. The library `lib/rhometta.metta` provides `rhometta:eval`, which defers a MeTTa term as the payload to be run at COMM; `rhometta:run`, which reduces a process to its quiescent outcome set; and `rhometta:transitions`, which returns one-step successors. It sits on `lib/rho.metta`, the strict-core constructor surface, and delegates all reduction to the engine. One consequence is central: when a deferred payload evaluates to *several* results, the COMM forks — one successor per result. Nondeterministic computation becomes a branching reaction, and `rhometta:run` returns one residual per branch (the *may-set*).

5.6 What reactions compute

The minimal cell. The smallest rhometta program is a single cell: a send carrying a deferred term beside a receiver that drops (runs) it. `rhometta-eval-cell` sends `(+ 2 3)` this way and the reaction leaves `5`; with a multi-result payload `(superpose (a b c))` the COMM forks into one branch per result, and `rhometta:transitions` confirms the fork is a single reduction step with three successors. Communication does computation, and result-multiplicity becomes branching.

A self-unfolding prover. Seed a *single* cell for a goal. Each COMM evaluates one backward-chaining step whose result is the cells for that goal's subgoals, so `rhometta:run` keeps reducing and the reaction network unfolds itself into a proof tree. Where a goal has several rules the payload has several results, so the COMM forks: `rhometta:run` returns one quiescent residual per complete proof tree (Figure 2) — two for the sample "can I bake a cake?" goal, while a single-rule goal stays confluent at one.

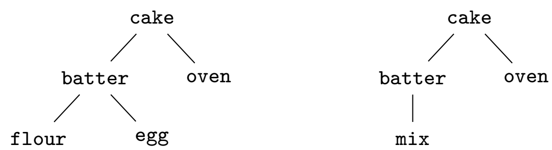


Figure 2: The self-unfolding prover's may-set for "cake": two complete proof trees, one per rule for `batter`. The trees are not enumerated up front — they are *grown* by reactions whose payloads plant their children.

Type inference as proof search. A typing judgment `(: proof (HasType e t))` is proved by reactions over typing rules, and the proof term is the derivation. In `rhometta-type-inference`, an overloaded `plus` yields a genuine may-set — an `Int` and a `Float` typing of `(plus one one)` — while `(plus x one)` with `x : Int` prunes the `Float` branch at the unification join. Proof terms stay pure, `(: (app pf pa pb) (HasType e t))` — a typing is crisp, so no truth value is attached. Each result is cross-checked differentially against a sequential chainer over the same knowledge base, which derives the same proof-term skeletons.

A truth value that is earned. Crisp proof terms are the right output for a typing, but some judgments are genuinely probabilistic, and there a reaction can carry evidence rather than certainty. In `rhometta-evidence-revision`, two agents each count their own observations of "ravens are black" into an opinion — a strength and a confidence — computed at COMM by a deferred payload. The two agents share two observations, so merging their opinions *naively* double-counts the shared evidence and overstates the confidence (0.889); an overlap-corrected revision discounts the shared stamps for the honest value (0.857), at the same strength. The revision can run after the parallel fan-in, or *at* the final rendezvous itself. This is the one place a truth value belongs — it is counted from data, not attached by hand.

Composition. These pieces compose: a further program runs an independent proof search, an overlap-corrected evidence revision, and an attention-spreading step concurrently, then rendezvouses their channel-gathered results into a single synthesis.

5.7 Cost-accounted reactions

A reaction is the natural unit of metering: each COMM is one indivisible piece of work. The `cost` profile makes that intuition operational — and it does so with money rather than a stopwatch. Processes are *signed*, channels carry *purses* of tokens, a reaction fires *only if it is paid for*, and what a run returns is not a number but a *receipt*: a causal, auditable record of every spend. The profile is the executable part of the programme to equip the ρ-calculus with compositional resource accounting [5, 6]. Its design commitments are locality of funding, causality from consumption, and an explicit separation between exact authority receipts and lossy numerical valuations.

Purses and funded reactions. In the `.rho` surface, `{P}alice` is the process `P` signed by `alice`, and `purse pay {alice : ()}` is a store on channel `pay` holding one token signed by `alice`. The funding rule is *located*: a COMM on `pay` consumes one token per participating endpoint signature, and those tokens must be found in purses *on `pay` itself*. There is no ambient bank account, and raw process data never funds anything. The smallest funded program has two signed endpoints and two purses (`tests/rhocalc_cost_run/split_tokens_basic.rho`):

```
{for ($m <- pay) {{0}cont}}alice | {pay!({0}payload)}bob
  | purse pay {alice : ()} | purse pay {bob : ()}
```

which reduces to `purse pay {()} | purse pay {()} | {0}cont`: each signature paid for its own endpoint, and the two purses are left empty beside the continuation. Now delete `bob`'s purse (`missing_token_quiescent.rho`) and the program is *quiescent with the send and receive still facing each other*: the reaction is not slowed, it is *disabled*. This is the first thing to internalize about the profile — cost is not an observer bolted onto the side; an unfunded reaction cannot fire. Because channels are quoted processes, a purse's location follows the calculus's own name equivalence rather than spelling: a purse on `@{*pay}` funds a reaction on `pay`, since the two names denote the same channel (`name_equivalence_quote_drop.rho`) — aliases of a channel are the same account.

Receipts are causal records, not counters. A single number would forget who paid, what was consumed, and what depended on what — and its intermediate readings would depend on which interleaving the scheduler happened to choose. Instead, every reaction emits an event record with four fields: a fresh event identifier; a *cause list* — the identifiers of the events that produced the exact occurrences this event consumed, with repetitions kept (consuming two outputs of event 0 yields the cause list `(0 0)`); a nonempty list of *funding* records naming the purse surface and head signature of each token spent; and the consumed signature itself. The two-reaction chain below (the first case of `tests/test_lts_rho_cost_causal_trace.metta`) shows the shape — event 1 consumes both a continuation and a purse tail produced by event 0, so its cause list holds two arcs to 0:

```
(lts:rho:cost:receipt
  ((lts:rho:cost:event 0 ()
     ((lts:rho:cost:funding chain spend)) spend)
   (lts:rho:cost:event 1 (0 0)
     ((lts:rho:cost:funding chain spend)) spend))
  (rho:cost:par
    (rho:cost:purse chain rho:cost:stack-empty)
    (rho:cost:signed rho:nil done)))
```

The algebraic reading is the load-bearing one. A receipt is a *located, measured pomset* of spend events: a partially ordered multiset whose order is generated purely by consumption — event e' follows e exactly when e' consumed something e produced. The engine's emission order is one linearization of that partial order. Any permutation of one fixed compatible matching yields the same target and receipt bag; the emitted sequence linearizes that matching's causal order. Alternative endpoint or funding matchings remain distinct branches. The Lean theorem `boundedBudgetedCausalPrefix_emission_linearizes` confirms that a budgeted prefix's emitted sequence linearizes its causal order. Locations make purses *fibers* over channels: funding never crosses a channel, and each per-location account is conserved (`CostPath.per_location_restriction_account_conservation`). Finally, the receipt itself carries no notion of "the cost": costs are *valuations* — monoid-valued maps out of the receipt (total spend, per-signature spend, critical-path depth) — so the receipt is the free object and each accounting policy is a homomorphism out of it, with the lawful combination rules derived rather than posited, in the sense of Knuth and Skilling's measure-assignment axioms [12]. Event identifiers are deliberately polymorphic in the same spirit: a local counter suffices in one process, and a hash-chained identifier can replace it at trust boundaries with the receipt algebra unchanged.

Honest budgets. Everything above is unbounded by default: `lts:rho:cost:steps` and `lts:rho:cost:causal-trace` enumerate exactly and never truncate silently. Bounded execution is strictly opt-in through `lts:rho:cost:causal-prefix`, which takes two separate allowances — reaction fuel and exact-cover search work — and returns a receipt together with a status that never lies: `quiescent` only after exhaustive search *proves* the residual has no funded successor; `fuel-exhausted` when the firing allowance ends first; `search-exhausted` when cover search stops before finding a successor *or proving absence*. Search exhaustion is never reported as quiescence: an out-of-fuel machine says "I ran out," never "there was nothing there." The distinction has teeth because funding search is a genuine exact-cover problem. A send whose signature product multiplies ten `coins`, beside twenty unit purses on the same channel, admits $\binom{20}{10} = 184{,}756$ distinct covers (`tests/test_lts_rho_cost_search_budget.metta`):

```
; Zero reduction fuel performs no exact-cover search, even when
; the latent cover family has C(20,10) members.
!(assertEqualToResult
  (test:prefix-status
    (lts:rho:cost:causal-prefix 0 0 (test:large-cover-state)))
  (lts:rho:cost:fuel-exhausted))

; The first exact occurrence cover is yielded without
; materializing the other 184,755 covers.
```

At fuel 0 the machine performs no search at all; with a small search allowance it yields the first cover lazily; and the unlimited frontier remains available and exact — all 184,756 successors, measured at about 11 s — combinatorial by design and by request, never by accident. A budget-monotonicity gate (59 paired runs at increasing allowances) checks that raising an allowance never flips a determinate verdict and never removes a receipt event: more fuel can only extend what is known.

Certified correspondence. The formal side lives in the `Cost/` theory in the Mettapedia repository: cost steps, paths, receipts, valuations, conservation laws, and a budgeted executor, proven equal to one another where they meet. The capstone `BudgetedPrefixPathRefinement` states that every budgeted machine run produces a prefix of the occurrence-bearing Lean executable `CostPath`, with the same receipt and the correct status, and its corollaries pin each verdict separately: sufficient allowances yield an actual path; exhaustive candidate failure — unlike bounded exhaustion — certifies quiescence; zero fuel and zero search-work report their own exhaustion and make no negative claim. The machine frontier is proven sound and complete up to structural equivalence (`costStep_sound_runtime`, `costStep_complete_runtime_up_to_struct`); these are the separate one-step correspondences with the declarative `CostStep` relation. The compositional theorem `budgetedCausalPrefix_exists_rhoLanguageDefRewritePath` closes the corresponding path square for the Lean executor: every publicly admitted budgeted run yields an occurrence-bearing runtime path, a declarative funded trace, and a rewrite path in the pure GSLT mechanically derived from `rhoCalc`, with exactly one pure step per runtime firing.

The pure-rho boundary is theorem-backed as well. The authored rho `LanguageDef` supplies the strict syntax and primitive COMM presentation; its compiled matcher, normalization, substitution, and supported COMM contracta are proved to agree with the independently established semantic development. The pure GSLT used for erasure is itself derived from that authored declaration; full equivalence of its complete contextual relation with the independently established relation remains a named theorem obligation. On the derived binder-safe fragment, initialization and funded reduction preserve scope; each declarative `CostStep` erases to one derived pure COMM, and each declarative funded or executable path erases to a derived rho rewrite path whose length is exactly its ordered demand-receipt length. The equivalence used here is pure structural congruence: no executable free-Drop step is admitted. The canonical configuration erasure is defined by quotient lifting rather than by a noncomputable representative choice. The pure-erasure binder-safety judgment covers exactly syntax observable by pure erasure. The separate public runtime-scope check validates every raw field, including purse surfaces, and is proved to imply the erasure-domain judgment. A rerunnable Lean audit prints the axioms of the exported results; they use only Lean's standard quotient and classical infrastructure, with no project axiom and no unfinished proof.

On the compiled side, 146 sequential/threaded prefix cases agree, and 308 compiled-C receipts replay as Lean derivations while two altered receipts are rejected. The complete runtime suite passes in the ordinary, thread-sanitized, and address/undefined-behavior sanitized builds. The differential harness (`scripts/rhocalc_cost_differential.py`) drives the same terms through the C engine and the Lean executor, including fuel-0 and fuel-1 boundary cases, with an out-of-fragment case correctly rejected. This remains bounded differential evidence about the compiled engine, not a universal theorem about C.

Parallel funded execution. Compatible reactions execute in OS threads in waves. Before a worker can fire, it atomically claims the exact endpoint occurrences and the exact funding-cover occurrences in a fixed resource order; a wave admits only pairwise disjoint claims. The sequential semantics therefore remains the reference, while the threaded implementation is tested as a legal parallel refinement: both threaded CLI modes, six contention and disjointness stress families, and the sequential/threaded prefix gate exercise same-channel disjoint reactions, competing funding covers, duplicate-looking occurrences, and causal receipt linearization. Receipt emission is an *optional observer* on this path: operational components are plain terms, causal provenance rides in a parallel producer vector, and a scripted transparency gate checks that a state-only run and a receipt-observed run consume identical resources and reach the same residual. These gates, together with thread and memory sanitizers, are implementation evidence; the Lean step/path theorem above supplies the declarative semantic boundary.

At the modeled parallel boundary, a list of enabled runtime candidates whose complete endpoint-and-funding source-index family is globally duplicate-free lifts to a `SelectedRuntimeWave`. Its witnessed source partition constructs an exact `CostMatching`, and every permutation of that fixed matching serializes as an ordinary declarative cost trace. This discharges the runtime-model wave-to-matching square under the occurrence-claim postcondition; the compiled-C correspondence remains the bounded differential, receipt-replay, and sanitizer evidence stated above rather than a universal theorem about C.

A strict source presentation and the corrected algebra. The profile implements funded rho and no Datalog architecture. Its strict syntax and primitive COMM presentation come from the authored `LanguageDef`; the compiled artifacts are proved to agree with the established pure semantics on supported COMM contracta, while the GSLT used for the cost-erasure theorem is mechanically derived from that authored declaration. Other implementations are bounded encoding references, and their executable free-Drop rule is not part of this profile.

The formalization also identifies why the original ordinary-monad faithfulness claim cannot be repaired by choosing a more cancellative signature monoid. Joint injectivity of a monoid multiplication together with a two-sided unit forces the carrier to be a subsingleton; words, multisets, and numerical costs all forget some factorization. The landed repair is an Atkey-style pre/post-indexed semantics [7]: a funded execution has complete source and target resource states, so incompatible funding transitions are untypeable. Exact authority receipts compose without erasure; price and budget totals are explicit, generally non-injective observations of those receipts. This is the resource-transition specialization of the wider graded and parameterized-effect account [8, 9]. The companion theory paper states the obstruction, the indexed laws, and the authority/observation factorization precisely [6].

The continued category and repeated Cost. Since the July development, the Lean side (`GSLT/LanguageDef/ContinuedCategory.lean`, `CostEndofunctor.lean`) treats Cost not as a one-off wrapping of rho but as a construction on *continued interactive GSLTs*: a language mechanically derived from one authored `LanguageDef`, equipped with one ordered interaction cut (a contact constructor plus program and environment operands with selected continuation positions), a computable canonical section of every typed open fiber, and a *position-sensitive continuation-retyping plan* under which every non-principal constructor receives a hereditarily wrapped copy while the two principals receive cut-aware base copies whose continuation parameter alone moves to the wrapped fiber. The generated funded language — the base/wrapped signature plus a small contact/signing/funding apparatus and exactly one whole-redex rewrite that consumes a funding-stack head and carries the tail through — is itself such an object, and the object-closure laws of the resulting endofunctor programme are largely discharged generically: bare-collection constructors of the generated language are provably non-principal at the next layer, the generated interaction envelope is stable under the next retyping, a single declaration-derived typing morphism validates the next layer's signature, and the typed transport along a retyping plan is proved over the minimal non-circular signature (theory, cut, plan). The remaining generated-object laws are active work with identified routes. Two corrections landed on the way and are recorded as compiled counterexamples rather than folklore: the global raw fiber-stability hypothesis of the earlier normalization architecture is *false* for rho (a mixed-colour reflective generator has a typed left endpoint whose canonical right endpoint is ill-typed), and its repair carries the entire equational theory on *typed* open carriers with proof-relevant region-tree elaboration — raw-to-typed lifting exists only edge-wise, with both endpoints certified in the same typed fiber. Exact typed canonical laws then cut out the full subcategory of *Cost-canonical* objects, which, ordered by collision-free structural canonical keys, is the honest domain of strict Cost_1. The theory paper develops all of this precisely [6]; the engine consequence is concrete: the same authored declaration that generates this profile's grammar and its single funded rewrite is the object the endofunctor consumes, so iterated cost-accounting is a construction on the specification, never a second hand-written engine. The public surface is `lib/lts/rho/cost.metta`; the engine is `src/rhocalc_core.c`. Runnable showcases ship with the engine in `examples/rho/` — a funded vending machine (an unfunded order left facing a live endpoint), a causal notary chain whose receipt is the provenance audit trail, an honest-budget meter exhibiting all three prefix verdicts and a two-cover spend, and a threaded settlement wave whose residual is byte-identical at every thread count — each with expected outputs checked by the test suite, alongside a documented walkthrough of the calculus, both surfaces, and the profile.

6 Cross-Runtime Benchmarks

Benchmark artifacts are labeled by dialect route. A row may be *shared* (one dialect-agnostic source), PeTTa-*native*, HE/CeTTa-*native*, mechanically translated, or hand-normalized after translation to expose the same algorithmic contract in both dialects. HE and PeTTa are different MeTTa dialects, so a real comparison is not "whatever file happens to run"; it is a named pair of artifacts plus a stated output contract. Timing rows in this section were measured on 2026-06-27 (the pre-GC tree), except the grounded genomic-inference rows (PLN evidence core, full WM-PLN raw-loader, mine→chain focus, and deep genomic backward chaining), which were re-receipted on 2026-07-02 on commit 9237afd; rows explicitly described as excluded, methodological, or unresolved carry their own status. The two heavy genomic rows (full WM-PLN raw-loader and deep genomic backward chaining) were re-measured on a quiescent host with the same binary at commit 9237afd; because the two engines are always compared within one session, the cross-engine ratios are the load-bearing claims. For rows whose single-run wall time is below 0.1 s, we report averaged full-process wall time over repeated invocations to avoid timer quantization. Output is verified identical across runtimes whenever a row is presented as a clean cross-runtime benchmark. So a reader can see precisely what is measured, each subsection below opens with the core of the program it runs; the full sources are linked inline and collected at `benchmarks/chaining_crossengine`.

Most of the inference benchmarks are driven by one small dependent-typed *backward chainer*. To prove a goal `(: $proof $theorem)` it either matches a fact in the knowledge base, or—for a rule application `($rule $prem)`—looks up the rule's type and recurses on its premise (the binary and n-ary cases follow the same shape):

```
(: bc (-> $a Nat $b $b)) ; KB, depth, goal -> goal
(= (bc $kb $_ (: $proof $theorem)) ; base case: it's a fact
   (match $kb (: $proof $theorem) (: $proof $theorem)))
(= (bc $kb (S $d) (: ($rule $prem) $theorem)) ; step: apply a rule
   (let* (((: $rule (-> (: $prem $t) $theorem))
           (bc $kb $d (: $rule (-> (: $prem $t) $theorem))))
         ((: $prem $t) (bc $kb $d (: $prem $t))))
     (: ($rule $prem) $theorem)))
```

Each benchmark below supplies its own typed rules and facts to this chainer (or, where noted, a forward chainer or a truth-value kernel).

6.1 Shallow Evaluation

Nil Geisweiller's typed backward chainer (the `bc` above) over 60 ground entities at depth 1 (20 researchers, 20 engineers, 20 artists). The rules are typed binary implications; for example, `r-mortal` combines a researcher fact with a curious fact to derive a mortal witness:

```
(: r-mortal (-> (: $r (researcher $x))
              (: $c (curious $x))
              (mortal $x)))
!(bc &kb (fromNumber 1) (: $proof (mortal $x))) ; -> 60 witnesses
```

All three runtimes produce identical 60 proof-carrying witnesses, e.g. `(: (r-mortal r01 r01c) (mortal r01))`.

Runtime	Time	vs. CeTTa	Output
CeTTa	4.0 ms avg	1×	60 witnesses
PeTTa	45.4 ms avg	11.3× slower	60 witnesses
HE	402 ms avg	100× slower	60 witnesses

This is a startup-dominated sanity row, not a scaling claim. It is useful because it exercises the typed proof-carrying surface while remaining small enough that translator and runtime drift are easy to spot.

6.2 Deep Proof Search: PeTTa Wins

Metamath proof search at depth 5 (2 axioms, proving $t = t$, 85,013 proof trees) reverses the performance ordering. The knowledge base is Geisweiller's `demo0` (axiom `a1`: equality is right-Euclidean; `a2`: 0 is a right identity; `mp`: modus ponens), and the goal is `t = t` (`01_nilbc/nilbc_th1.metta`; glyphs shown ASCII here):

```
(: a1 (-> (: $t term) (: $r term) (: $s term)
        (-> (= $t $r) (-> (= $t $s) (= $r $s))))) ; right-Euclidean
(: a2 (-> (: $t term) (= (+ $t 0) $t))) ; 0 right-identity
(: mp (-> (: $maj (-> $P $Q)) (: $P wff) (: $Q wff)
        (: $min $P) $Q)) ; modus ponens
!(bc &kbh (fromNumber 5) (: $prf (= t t))) ; prove t = t
```

Runtime	Time	vs. PeTTa	Status
PeTTa	7.60 s	1×	85,013 proofs
CeTTa	36.42 s	4.8× slower	85,013 proofs
HE	>120 s	timeout	killed

PeTTa's Prolog search remains decisively better on deep branching proof search; the current rerun widens the gap to roughly 4.8×. HE still cannot finish within 120 s.

6.3 Forward Chaining at Scale

Nil Geisweiller's propositional-calculus forward chainer at depth 4 uses a solution space that is extended in place by forward application of modus ponens:

```
(= (fs-add-to-ss $rb $ss)
   (with-space-snapshot $ss0 $ss
     (match $ss0 (: $x1 $a1)
       (match $ss0 (: $x2 $a2)
         (match $rb (: $f (-> $a1 $a2 $b))
           (add-atom $ss (: ($f $x1 $x2) $b)))))))
!(count-atoms &ss)
```

The current CeTTa rerun of `tests/nil_pc_fc_d4.metta` completes in 100.86 s and reports 8.19M accumulated results after the four forward passes. We do *not* reuse the older "11.7M generated pairs" sentence as benchmark evidence, because the current HE lane does not time out on this driver; it errors immediately (`match expects a space as the first argument`). So the forward row remains a useful workload, but it is not yet paper-ready as a clean cross-runtime benchmark.

6.4 Translator Validation

A PeTTa-native benchmark using `prog1` (PeTTa-specific construct) was translated PeTTa→HE and run on CeTTa. The translator converts `prog1` to nested `let` with fresh variables:

```
; PeTTa original:
(= (run-query) (prog1 (bc &kb ...) (println! "done")))
; Translated to HE:
(= (run-query) (let $__tr_result_1 (bc &kb ...)
                 (let $__tr_discard_2 (println! "done")
                   $__tr_result_1)))
```

The current source file `benchmarks/bench_backchain_petta_native.metta` produces 30 proof-carrying witnesses on native PeTTa, and the current translated artifact produces the same 30 witnesses on both CeTTa and HE:

Runtime	Time	vs. CeTTa	Output
CeTTa	3.7 ms avg	1×	30 proofs
PeTTa	44.8 ms avg	12.1× slower	30 proofs
HE	319 ms avg	86× slower	30 proofs

This row is a translator benchmark, not just a syntax example: the observable runtime behavior survives translation.

6.5 PLN Evidence Aggregation over Genomic KB

To benchmark inference over real data, we run PLN evidence aggregation over the Rejuve genomic knowledge base (chr16 pilot: 41 GWAS SNPs, 183 SNP–gene pairs, 3 evidence channels). Each pair carries eQTL tissue counts (GTEx, 49 tissues), ABC enhancer–gene contact counts, and a regulatory presence flag. The PLN evidence quantale combines channels via revision (`evidence-hplus`), producing a calibrated SimpleTruthValue per pair. The evidence operations are Lean-verified: `power`, `toLogOdds_power_mul`, and `power_power_inv` are proved in `Mettapedia/Logic/BinaryEvidence.lean`.

The core arithmetic is ordinary MeTTa over numbers:

```
(= (Truth_ModusPonens (stv $f1 $c1) (stv $f2 $c2))
   (stv (+ (* $f1 $f2) (* 0.02 (- 1 $f1))) (* $c1 $c2)))
(= (Truth_Revision (stv $f1 $c1) (stv $f2 $c2))
   (let* (($w1 (Truth_c2w $c1)) ($w2 (Truth_c2w $c2)) ($w (+ $w1 $w2))
          ($f (/ (+ (* $w1 $f1) (* $w2 $f2)) $w)) ($c (Truth_w2c $w)))
     (stv (min 1.0 $f) (min 1.0 $c))))
```

The benchmark is dialect-agnostic (shared MeTTa, no translation needed). The portable `benchmarks/bio_pln` evidence-score drivers produce numerically identical 183 scored pairs on CeTTa, PeTTa, and HE:

Runtime	Time	vs. CeTTa	Output	Status
CeTTa	10.9 ms avg	1×	183 bio-scores	identical
PeTTa	58.1 ms avg	5.4× slower	183 bio-scores	identical
HE	7.56 s	696× slower	183 bio-scores	identical

Scores carry genuine biological signal: `rs9923732`→`ENSG00000166455` scores strength 0.7348066298342542 (133/181) and confidence 0.9945054945054945 (181/182), while most pairs score 0.04 (minimal evidence). All three engines reproduce every one of the 183 scores to full numeric precision on the portable drivers; the earlier non-portable driver diverged only because upstream HE integer-divided the evidence ratio to zero and PeTTa left imported evidence-library calls unreduced. Exact three-engine numeric agreement is this row's validation story.

What we no longer claim here. Earlier drafts included a synthetic scaling table from 143 to 1.4M atoms. We still do not treat that older synthetic table itself as benchmark evidence in the paper. What *is* now grounded again is the true full raw-loader checkpoint lane: the fresh CeTTa and PeTTa reruns both execute the same TFBS-inclusive 1,399,997-line data tier and agree on the labeled observable contract (`9` seed targets, `9` contextual targets, `85` disease proofs, `34` supported-drug proof witnesses, and `8` unique supported-drug hypotheses). The explicit proof-count vs. hypothesis-count split matters here: the `34` traces are genuinely multiple proof routes collapsing to `8` unique `(gene,disease,drug,tissue)` hypotheses, not a stale counting mistake.

Fresh timings on this grounded full row are 16.4 s on CeTTa and 44.5 s on PeTTa (about 2.7× faster). Upstream HE does not currently run this loader shape directly: its `metta` runner rejects the relative-file `import! ./support/...` surface as an illegal module name, so the HE column for this row remains a loader blocker rather than a performance number. The 183-pair core therefore remains the grounded small PLN row, while the full TFBS-inclusive checkpoint is now the grounded large WM-PLN row.

6.6 Genomic Pattern Mining

To exercise real algorithmic depth, we run the Hyperon pattern miner [1] on real GTEx eQTL data, discovering surprising co-regulation patterns—pairs of SNPs or genes that share eQTL evidence more often than expected by independence. The miner is run unmodified from github.com/iCog-Labs-Dev/hyperon-miner; its core loop alternates `build-specialization` (extend a candidate pattern by one link) with a `surprisingness` score that keeps only patterns whose support exceeds the independence expectation. The data is extracted from the Rejuve genomic KB (338M Prolog lines, 57 GB) and discretized into categorical attributes (tissue breadth, effect size) for pattern mining.

The current audit does *not* reuse the older cross-engine mining speed table as benchmark evidence, and no current in-tree driver re-emits the earlier specialization-core atom count, so we do not carry that number forward. We keep the pattern miner as a methodological workload. Its portability boundary in this revision is HE's stateful use of `superpose`: a remove/add pattern that mutates a shared space in the middle of a `superpose` branch:

```
(superpose ((remove-atom $specspace $spec1)
           (add-atom $specspace $spec1)
           (remove-atom $specspace $spec2)
           (add-atom $specspace $spec2)
           (remove-atom $specspace $spec3)
           (add-atom $specspace $spec3)))
```

Current CeTTa preserves all three atoms produced by this pattern, while current HE's interleaved side-effect evaluation drops the third—the same atom-loss hazard analyzed in Section 9 (the pattern miner). So we keep the miner as a portability/debugging result, not as a fresh cross-engine speed table.

6.7 Backward Chaining over Mined Patterns

The mined co-regulation patterns serve as typed inference rules for Geisweiller's `nilbc` backward chainer (the same dependent-typed `bc` listed at the start of Section 6), closing the *mine→chain* pipeline. The knowledge base combines:

- 100,000 real GTEx eQTL links (40,484 SNPs × 14,043 genes),
- 13 co-regulation patterns and 7 co-expression patterns (mined, surprisingness > 0.98),
- 3 typed PLN rules: *direct-target* (eQTL → target), *coreg-transfer* (co-regulation + target → target), and *coexpr-transfer* (target + co-expression → target).

The three rules are typed implications fed to the `bc` chainer of Section 6; *direct-target* reads an eQTL fact, *coreg-transfer* chains a co-regulation edge onto an existing target:

```
(rule-sig dt (-> (: $e (eqtl $snp $gene)) ; direct-target
              (target $snp $gene)))
(rule-sig ct (-> (: $e1 (coreg $snp1 $snp2)) ; coreg-transfer
              (: $e2 (target $snp2 $gene))
```

```
            (target $snp1 $gene)))
!(bc &kb (fromNumber 2) (: $proof (target rs10000544 $gene)))
```

Bridge base cases let `nilbc` consume untyped data atoms (`eqtl-link`, `co-regulate`) directly, avoiding a format conversion pass over the 100K-atom KB.

On the portable `benchmarks/mine_chain_focus` focus-summary drivers, CeTTa and PeTTa agree exactly on the distinct direct and depth-2 target genes for three representative query SNPs (the raw proof rows dedupe by target gene to these counts):

Query SNP	direct targets	depth-2 targets	novel
rs10000544	9	12	3
rs10000620	5	5	0
rs10000960	6	8	2

The five novel depth-2 targets are:

Gene	Symbol	Via SNP	Evidence path
ENSG00000150961	SEC24D	rs10000795	`(ct cr3 (dt eq37))`
ENSG00000172399	MYOZ2	rs10000795	`(ct cr3 (dt eq39))`
ENSG00000178636	MRPL42P1	rs10000726	`(ct cr1 (dt eq30))`
ENSG00000248394	FOSL1P1	rs10001048	`(ct cr25 (dt eq87))`
ENSG00000273077	(lncRNA)	rs10001048	`(ct cr25 (dt eq89))`

The proof terms are machine-checkable evidence chains: `(ct cr3 (dt eq37))` reads "co-regulation transfer via `cr3` (rs10000544↔rs10000795), applied to direct eQTL `eq37` (rs10000795→SEC24D)." On these focused three-query drivers the total wall-clock time is 0.13 s on CeTTa versus 4.27 s on PeTTa. We therefore keep the novel-gene discovery story, but we do *not* reuse the older proof-count/time row as benchmark evidence in this revision.

Biologically, the novel discoveries include MYOZ2 (hypertrophic cardiomyopathy gene, OMIM CMH16) and SEC24D (Cole-Carpenter bone syndrome), both on chr4q26—suggesting a trans-regulatory link from a metabolic locus (PDE5A/FABP2) to cardiac and skeletal gene targets via shared cis-regulatory architecture.

The older "all 7 query SNPs plus reverse queries" timing line is not used as a fresh paper benchmark here; the audited evidence in this revision is the focused cross-runtime driver above.

Complete benchmark ladder. The following table summarizes all benchmarks, including both genomic inference (where CeTTa wins) and pure search (where PeTTa wins):

Benchmark	Scale / note	CeTTa	PeTTa	HE
Shallow typed BC	60 proof witnesses	4.0 ms avg	45.4 ms avg	402 ms avg
Translator (`prog1`)	30 proof witnesses	3.7 ms avg	44.8 ms avg	319 ms avg
PLN evidence core	183 scored pairs	10.9 ms avg	58.1 ms avg	7.56 s
Full WM-PLN raw-loader	1,399,997 lines, 34 proofs / 8 hyps	16.4 s	44.5 s	import block
Mine→chain focus	3 query SNPs, 5 novel genes	0.13 s	4.27 s	—
Deep 17-phase genomic BC	1,411,430 hypotheses	46.33 s	64.05 s	—
8-Queens (direct shared lane)	92 solutions	8.22 s	0.05 s	2:08.77
Deep proof search	85,013 proofs	36.42 s	7.60 s	>120 s

Peak resident memory on the two heavy rows. On the full WM-PLN raw-loader, CeTTa uses 2.10 GiB against PeTTa's 4.75 GiB (CeTTa lower). On the deep 17-phase genomic BC the direction reverses: CeTTa uses 14.5 GiB against PeTTa's 4.85 GiB—about 3.0× PeTTa's footprint, the memory-multiplicity cost of CeTTa's answer materialization on the high-fan-out row.

CeTTa still wins on several real inference rows, but the revised ladder is intentionally smaller than earlier drafts. In the 2026-06-27 audit we excluded the historical cross-engine miner speed table, the old full-1.4M WM-PLN scaling table, and BTree from this cross-engine ladder because at that point the tree either lacked a stable current cross-runtime driver for them or exposed a correctness/comparability issue. The tilepuzzle row has since been rerun and is reported separately as a search benchmark and GC capacity result in Section 7.2. The benchmark rows shown here are only those rerun directly in that audit.

6.8 Cross-Engine Chaining Suite

To ground the inference-side comparison on community-authored code rather than bespoke benchmarks, we assembled a cross-engine chaining suite that builds matched cross-engine rows from the same chainer families on CeTTa (`-lang he`) and PeTTa, with a per-row cross-engine contract (every result is validated against the other engine's output, never against CeTTa's own output). The sources are Geisweiller's dependent-typed DTL chainers and Metamath `demo0` proofs (github.com/ngeiswei/chaining), Treutlein's `PeTTaChainer` (github.com/rTreutlein/PeTTaChainer), and the PeTTa engine and its PLN library (github.com/patham9/PeTTa). The suite, generators, and driver are in the CeTTa repository under `benchmarks/chaining_crossengine`. In this paper revision we use the suite as audited reproducibility material rather than as an extra source of unevaluated claims: the benchmark rows retained in the paper are only the subset rerun directly in this audit.

Multi-relational deep chaining. The strongest current multi-relational row is the 17-phase snapshot `tests/bench_bio_bc_deep_snapshot_20260327.metta`. It extends the mined-pattern chainer with pathway membership, same-pathway transfer, disease associations, compound–disease treatment links, and compound–gene binding links. The full variant-to-drug row is still a typed backward-chaining program:

```
(rule-sig dg (-> (: $e1 (target $snp $gene))
                 (: $e2 (has-disease $gene $disease))
                 (disease-reach $snp $disease $gene)))
(rule-sig td (-> (: $e1 (disease-reach $snp $disease $gene))
                 (: $e2 (treats $compound $disease))
                 (drug-hypothesis $snp $compound $disease $gene)))
```

The current rerun produces exactly **1,411,430 hypotheses** across 17 query phases, identical on both runtimes, with the bucket breakdown:

Hypothesis bucket	Count
`pw-target`	613,978
`deep-pw`	796,964
binding	11
disease	11
drug	210
deep-drug	240
pathway-reach	16
Total (17 phases)	1,411,430

Runtime	Wall time	Peak RSS	vs. PeTTa	Output contract
CeTTa	46.33 s	14.5 GiB	1.38× faster	1,411,430 hypotheses
PeTTa	64.05 s	4.85 GiB	1×	1,411,430 hypotheses

The current timings still favor CeTTa, but the table makes the memory tradeoff explicit: this high-fan-out row is one where CeTTa materializes many answers and pays roughly 3× PeTTa's peak footprint. The textbook chain rs10000544 → PDE5A → hypertension ← Sildenafil composes three independent data sources (GTEx, Hetionet, DrugBank) by typed backward chaining with proof terms at every step.

Dependent binder encoding. PLN rule declarations use `(rule-sig name type)` instead of `(: name type)` to avoid a current HE spec limitation: the `(: $e T)` syntax in arrow-domain positions is treated as a literal expected type, not a dependent binder, causing `BadArgType` on both HE and CeTTa (see Section 10). The `rule-sig` encoding is semantically equivalent on all runtimes.

6.9 Four-Engine Study: JeTTa and MeTTaScript Join the Ladder

A 2026-07 study extended the ladder with JeTTa (a Kotlin/JVM ahead-of-time compiler for MeTTa) and, via its upstream slide deck, MeTTaScript (TypeScript) [10, 11]. Every timed run was gated on answer correctness; a fast wrong answer counts as a failure. Three findings matter for CeTTa's roadmap:

- **Competence zones, not a single ranking.** JeTTa dominates deterministic arithmetic recursion (its compiler auto-memoizes pure recursion: naive Fibonacci runs in $O(N)$); PeTTa dominates unindexed nondeterministic search (8-Queens roughly 150× over CeTTa) and is the only engine with production tabling and bignums; CeTTa keeps near-zero startup and wins indexed knowledge-base queries (the mined-chain workload roughly 26× over PeTTa); on the heaviest typed proof search in the study (the 85,013-proof Metamath depth-5 run) PeTTa completes in 13.0 s under its shipped 8 GiB stack configuration versus 62.7 s for CeTTa — the 4.8× margin matching the earlier audited row. An initial rerun under SWI-Prolog's 1 GiB *default* stack overflowed and was briefly misrecorded as an engine failure; the harness, not the engine, was at fault.
- **The per-call and sharing taxes, measured.** On naive Fibonacci the per-call interpretation costs were approximately 0.23 μs (PeTTa, WAM) versus 9.6 μs (CeTTa tree walk) —

a $\approx$ 42× machine tax — while the tabling three-way showed declared tabling and auto-memoization both answering `fib 46` in 0.10 s where the untabled tree walk extrapolates to tens of hours. These are the evidence rows behind the abstract machine hypothesis (Appendix on MAM) and the call-keyed sharing direction in Prime.

- **Methodological hazards.** Auto-memoization silently turns repetition-scaled benchmarks into cache-lookup measurements; one engine returned fast *wrong* sums through silent 32-bit wraparound; and a harness that launches an engine without its shipped resource configuration (here, SWI-Prolog's 1 GiB default stack in place of PeTTa's configured 8 GiB) manufactures failures that read as engine limits. All three hazards argue for correctness-gated, semantics-aware benchmarking rather than wall-clock tables.

7 Performance Analysis

7.1 Current profiling status

Valgrind reruns on the grounded targets, measured on the pre-GC tree (2026-06-27), split into one success mode and one blocker. Massif reruns succeeded on both the restored 8-Queens row and the full TFBS-inclusive WM-PLN row. On direct 8-Queens, the fresh peak useful heap is 173.5 MB, dominated by arena allocation (`arena_claim_block`), `atom_symbol_id`, `atom_deep_copy_impl`, and the typed dispatch / match / eval path. On the full WM-PLN row, the fresh peak useful heap is 2.09 GB, dominated first by import-time term-universe growth and symbol interning (`cetta_realloc`, `term_universe_insert_new_record`, `tu_intern_symbol`, `parse_metta_file_ids`), then by query-time delayed result storage (`stree_insert`, `native_query`). By contrast, fresh Callgrind reruns on both grounded targets do *not* yet yield a usable callgraph file: Valgrind hits a `brk segment overflow` before emitting a nonempty callgrind output. We therefore report fresh Massif measurements here, but continue to withhold exact Callgrind percentages on the current build.

7.2 Tilepuzzle: PeTTa comparison and GC capacity repair

The 8-puzzle breadth-first search (BFS) remains a useful WAM-class stress test. The normalized mathematical contract is the size of the solvable parity class reachable from the solved board, including the start state: $9!/2 = 181{,}440$ board configurations. The CeTTa port (`benchmarks/test_tilepuzzle.metta`) is a hand-normalized HE/CeTTa counterpart of the PeTTa example. The algorithmic contract is the same BFS over the solvable parity class, but the supporting surfaces differ: PeTTa uses its `lib_datastructures` queue and `add-unique-or-fail` visited-set idiom, while CeTTa uses queue-space, hash-space, and the HE-portable zero-results idiom `case(collapse(match))`. It now returns the labeled board-count result `(tilepuzzle-reachable-state-count including-start 181440)`. The PeTTa native example (`examples/tilepuzzle.metta`) retains its historical self-check of 181,441 as its native observable. The comparison below separates that observable from the normalized board-count contract; it does not claim an extra reachable board.

Artifact	Native observable	Contract	Time	RSS
CeTTa pre-GC port	`[181440]`	181,440 boards	26.41 s	5.86 GiB
CeTTa post-GC port	`including-start 181440`	181,440 boards	8.80 s	233 MiB
PeTTa native	`test ... 181441`	181,440 parity class	1.40 s	121 MiB

This row should therefore be read in two ways. As a cross-runtime search benchmark, PeTTa still wins decisively: the Prolog/WAM lane is about 6.3× faster than post-GC CeTTa on the native example and uses about half the peak memory. As a CeTTa capacity regression, however, the row is repaired: the full board-count contract now completes in single-digit seconds instead of exhausting the old evaluation-arena envelope.

The CeTTa search runs as a single top-level form, and each BFS step builds a fresh evaluation environment. Before the fix, those per-step environments were never reclaimed inside the evaluation arena, so memory grew linearly in the number of visited states. The fix is a tail-safe garbage collector for the evaluation arena, on by default. It (i) skips environment merging for *closed* queries—queries with no free variables escaping to the caller—and (ii) at tail-call-safe points evacuates the live root set (the current term, its environment, and its evaluation type) into a small survivor arena, then resets the evaluation arena to a per-call anchor; the survivor arena is itself reset at each top-level form boundary. Because only the roots that outlive a step are copied forward, the transient per-step environments are freed in bulk instead of accumulating.

With the collector on (commit 9237afd), the full 8-puzzle BFS completes well within its envelope, and the reclaimed memory is stable under the collection-budget spot checks:

Full run, collector on (commit 9237afd)	Value
States visited	181,440
Wall time	8.80 s
Peak resident memory, default budget	233 MiB
Peak resident memory, 1 MB budget	178 MiB
Eval-arena memory reclaimed	5,704,882,536 B (≈5.3 GiB)
Survivor-arena peak	135,432 B (≈132 KiB)
Survivor resets	1

At the 1 MB budget the run returns the identical 181,440-state payload as at the default budget (with even lower peak memory), so the collector preserves the computed result rather than changing it. It is differentially validated, not verified—a collector-on versus collector-off comparison that requires identical outputs on every case:

Collector-on vs. collector-off	Cases	Agree
Fast regression corpus	406	406
ASan/UBSan + provenance asserts	404	404

Two further ASan/UBSan cases are skipped only because the collector-off reference exceeded its bounded time limit.

7.3 Structured Space Kinds

CeTTa now supports three ordered space kinds beyond plain atomspaces:

Kind	Discipline	Use case
`queue`	FIFO push/pop	BFS frontiers, work queues
`stack`	LIFO push/pop	DFS, continuation stacks
`hash`	O(1) membership	Visited sets, deduplication

These are created via `(new-space queue)`, `(new-space hash)`, etc., and support universal pattern matching through the standard `match` API. The tilepuzzle uses queue-space for its BFS frontier and hash-space for visited-set deduplication. The current paper claim for that workload is the post-GC capacity baseline: 181,440 reachable boards in 8.80 s at 233 MiB peak RSS (Section 7.2). Older plain-atomspace versus queue/hash memory comparisons are kept as historical diagnostics rather than current benchmark evidence.

7.4 Architecture: Three Machines

Profiling and design analysis (informed by GPT-5.4 Pro architectural review) identifies three semantically distinct machines that should not share mutable state:

1. **Local search machine.** Branch-local bindings, trail/rollback, choice frames, scratch arenas. WAM-inspired: cheap undo via watermarks, not clone/free churn. This is the next implementation priority.
2. **Indexed shared-space machine.** Persistent PathMap/MORK-backed spaces, snapshot/epoch discipline, conjunction query planning, leapfrog triejoin. Deterministic and committed—no backtracking through the index.
3. **Type/elaboration machine** (future). Scoped binders, metavariables, constraint solving, delayed assignments. Separate from runtime bindings.

Language profiles (`he_compat`, `he_extended`, `he_prime`, future `petta`) are compositional assemblies over these machines, not ad-hoc forks in the evaluator.

7.5 Faithful Backend Obligations

The indexed shared-space machine needs a sharper correctness contract than "the backend is fast and seems to return the same answers." Recent Lean work factors the backend story into three explicit runtime seams:

1. **Candidate selection.** The index narrows candidate atoms, while CeTTa's native matcher performs exact variable-sensitive matching. This is the two-phase architecture now validated by concrete selector theorems over PathMap tries and by a concrete matcher instantiation on the HE side.
2. **Canonical storage.** Storage may quotient results to canonical support, but it must preserve co-reference. Repeated source variables must remain repeated after storage and materialization; distinct variables must remain distinct.
3. **Revision-based caching.** Cached query answers are valid only across mutations that preserve the relevant support boundary. Duplicate adds or high-count decrements may preserve cache validity; first adds and last removes must invalidate. Variant-normalized cache keys must also preserve right-hand-side answers and result shape under materialization.

This yields a practical division of labor for the PathMap/MORK backend:

1. PathMap stores canonical support and performs prefix-based candidate narrowing;
2. the native matcher remains the source of truth for exact bindings and binder hygiene;
3. count metadata, not trie-key distortion, carries bag semantics.

Verified runtime contracts. Recent Lean work extends the backend correctness story with three new contract files:

- `CeTTaRuntimeContracts.lean` (20 theorems): canonicalization injectivity and BEq preservation (`term_canon.c`), search-context save/rollback correctness (`search_machine.c`), effect classification safety (`runtime_effect_classes.json`), and profile hierarchy (`session.c`). Each theorem maps to a concrete refactoring risk it reduces.

- `IncrementalTableSemantics.lean` (15 theorems): specifies partial-table soundness, answer monotonicity, and completion for future tabling in `table_store.c`. Any correct implementation must satisfy `TableEntry.Sound` (partial reads are genuine) and `TableEntry.Exact` (completed tables equal `queryEquations`).

- `ProducerChoicePoint.lean` (10 theorems): formalizes the search-machine save/rollback/nested-choice-point protocol with combined bindings+scratch state.

Crucially, CeTTa's imported backend is binding-correct not through trusted direct bridge bindings, but because candidate narrowing is followed by seeded native rematch (`space_subst_match_with_seed`). The theorem `imported_seedRematch_binding_parity` in `SeedRematchContract.lean` formalizes this: when imported candidates are sound, the two-phase query produces the same result set as direct matching.

Positive example. Adding an existing atom to a counted space may increment multiplicity without changing trie support or invalidating unrelated cached queries.

Negative example. Letting the backend bridge perform full unification over serialized query terms invites exactly the binder-hygiene failures that arise when canonicalization preserves shape but breaks co-reference.

These obligations are now mapped one-for-one to runtime seams: `space_match_backend.c` for candidate selection, `term_universe.c` for canonical storage, and `table_store.c` for cache invalidation. The companion note *Faithful Backends for HE MeTTa* develops the full proof story, including the variant-query and binding-cache lemmas; the present paper uses those results to justify the architecture of CeTTa's indexed shared-space machine.

7.6 MM2 Integration

MM2 is a deterministic, priority-scheduled, forward-chaining term rewriting language implemented by MORK. Its programs are not a separate execution layer—they are atoms in MORK-backed spaces, stepped by MeTTa via `step!`. The hybrid interaction requires no special protocol: MeTTa creates spaces, adds data, steps MM2 rules, and reads results through ordinary `match`.

```
!(bind! &ws (new-space mork))
!(add-atom &ws (exec (0 step)
  (, (edge $x $y) (edge $y $z))
  (O (+ (path $x $z)))))
!(add-atom &ws (edge a b))
!(add-atom &ws (edge b c))
!(step! &ws)
!(match &ws (path $x $y) ($x $y)) ; -> (a c)
```

Three import paths exist: `.mm2` file import (`import!` or `include` into a target space, which auto-enables the MORK backend), inline `add-atom` of exec rules from MeTTa, and compiled `.act` artifact loading via `mork:open-act`.

Because MM2 rules are ordinary atoms, MeTTa can pattern-match over them as data—inspecting priorities, rewriting source/sink templates, or extracting rules for analysis:

```
!(match &ws (exec $p $src $snk) (rule $p))
```

CeTTa's `mm2_lower.c` handles bidirectional surface↔IR lowering (`exec`↔`mm2_exec`, `,`↔`mm2_pattern_and`, etc.), preserving round-trip fidelity between MM2 surface syntax and CeTTa's internal representation.

8 The MeTTa Dialect Landscape

8.1 Shared Fragment

85% of MeTTa programs use only shared constructs (`let`, `let*`, `match`, `if`, `case`, `bind!`, `add-atom`, `new-space`). These run identically on all three runtimes without translation. The Metamath proof search benchmark is dialect-agnostic. This shared fragment is essentially the core of the upstream MeTTa specification, of which HE, PeTTa, and CeTTa are three implementations.

8.2 Translated Constructs

Direction	Construct	Translation
HE→PeTTa	`chain`	`let`
	`collapse-bind`	`collapse`
	`superpose-bind`	`superpose`
	`nop X`	`let $fresh X ()`
	`function (return X)`	`X` (unwrap)
PeTTa→HE	`progn A B`	`let $fresh A B`
	`prog1 A B`	`let $r A (let $d B $r)`
	`@<`	`<s`

8.3 Semantic Divergence: Constraint Store vs. Independent Scope

The translation preserves semantics on the *pure fragment* (ground terms, or variables scoped by `let*`). A genuine divergence exists for free variables in non-deterministic branches:

```
(= (deduce (And $a $b)) (And (deduce $a) (deduce $b)))
```

When `deduce` returns via `match &self` with a free variable `$x`, HE/CeTTa bind `$x` consistently across both branches of `And`, while PeTTa gives each branch its own copy.

This divergence has a precise theoretical characterization. Saraswat et al. [2] showed that concurrent constraint languages based on Tell-and-Ask over a shared store satisfy a *monotonicity property*: constraints accumulate monotonically, and any reduction possible in a later configuration was already possible in an earlier one (their Proposition 2.1). HE/CeTTa's `match &self` operates on a shared constraint store—bindings produced by one branch propagate to all others via the shared space. PeTTa's Prolog backend evaluates branches with independent variable scopes, closer to what Saraswat et al. identify as the "read-only variable" model of Flat Concurrent Prolog, which they explicitly note does *not* possess the monotonicity property.

The depth-1 standard form. Saraswat et al. define a clause to be in *standard form* when each atom in the body is depth-1 (no shared variables between conjunction branches). Nil Geisweiller's typed backward chainer uses exactly this form: `let*` threads variable bindings explicitly between premises, eliminating implicit sharing. This is why the typed chainer works identically on all three runtimes—it stays within the standard-form fragment where the constraint-store and independent-scope semantics coincide.

Lean formalization. The `Translatable` and `PureTranslatable` predicates in `HEPeTTaSound.lean` characterize precisely this fragment: atoms where `atomToPattern` succeeds and produces `isMatchCorrect` patterns (no lambda, subst, or collection nodes). The sorry-free proof establishes that equation-matching semantics are preserved under translation on this fragment. Extending the proof to the shared-variable conjunction fragment would require formalizing the monotonic constraint-store semantics of HE's `match &self`—a natural target for Phase 3 of the Lean proof.

9 Non-determinism and Mutable State

A fundamental specification gap in MeTTa concerns the interaction of non-deterministic evaluation with mutable space operations. When `let $x (match &db ...) (add-atom &result (stored $x))` executes with multiple match results, what happens to the shared `&result` space?

9.1 The Problem

Consider:

```
!(bind! &db (new-space))
!(add-atom &db (item A))
!(add-atom &db (item B))
!(bind! &result (new-space))
!(let $x (match &db (item $y) $y)
   (superpose ((remove-atom &result (stored $x))
              (add-atom &result (stored $x)))))
!(get-atoms &result)
```

HE returns `[]`—all atoms lost. CeTTa returns `[(stored A) (stored B)]`—correct. The root cause: HE interleaves side effects from different non-deterministic branches, so `remove-atom` from one iteration cancels `add-atom` from another. CeTTa evaluates each iteration's body to completion before starting the next. This is the exact pattern used by the Hyperon pattern miner's `build-specialization`, explaining why the miner produces empty results on HE.

9.2 Four Possible Semantics

We analyzed four options across eight representative examples (accumulation, logging, constraint checking, search with backtracking, speculative execution, order-dependent processing, accidental non-determinism, and the real miner code):

Option	Wins	Loses	Character
Sequential (CeTTa)	6/8	1/8	accumulation-friendly
Interleaved (HE)	0/8	4/8	always worse than sequential
Forked (copy-on-write)	1/8	4/8	search-friendly
Illegal (UB)	1/8	5/8	safest, least ergonomic

Sequential evaluation loses only on *search with backtracking*, where each branch needs isolated state. Interleaved evaluation is strictly dominated—it provides no advantage over sequential while introducing race conditions on shared mutable state.

9.3 `with-space-snapshot`: The Escape Hatch

For the one case where sequential semantics is wrong (search with mutable state), CeTTa provides `with-space-snapshot`:

```
(= (fs-add-to-ss $rb $ss)
   (with-space-snapshot $ss0 $ss
     (match $ss0 (: $x1 $a1)
       (match $ss0 (: $x2 $a2)
         (match $rb (: $f (-> $a1 $a2 $b))
           (add-atom $ss (: ($f $x1 $x2) $b)))))))
```

This pattern, from Geisweiller's propositional forward chainer, reads from a frozen snapshot `$ss0` while writing to the live space `$ss`—the standard *semi-naive* discipline for stratified forward inference. This is a CeTTa extension; HE does not currently implement it.

9.4 Proposed Spec Clarification

The MeTTa spec [1] uses sequential Python pseudocode (line 368: a list comprehension over results) that implies sequential side-effect ordering, but does not explicitly require it. We propose:

> *When a non-deterministic expression produces multiple results and those results are bound via `let`/`chain`/equation dispatch, the body for each result is evaluated to completion—including all side effects—before the body for the next result begins. To evaluate branches with isolated state, use `collapse` first, then iterate explicitly.*

A future `snapshot-space` primitive would provide clean copy-on-write semantics for search/backtracking without baking merge semantics into the language core.

10 HE′: Dependent Telescope Extension

Both HE and CeTTa treat `(: $e T)` in arrow-domain positions as a literal expected type, not a dependent binder. This means Curry-Howard backward chainers cannot store rule types as `(: dt (-> (: $e (eqtl $snp $gene)) (target $snp $gene)))` in `&self` without triggering `BadArgType`. This is a current HE spec limitation, not a CeTTa bug.

We formalize an explicit extension, HE′ (he_prime), as a *telescope interpretation* of arrow domains. The extension is exactly one function: `splitDomain`, which recognizes `(: $x T)` as a dependent binder instead of a literal. Everything else (matching, bindings, space operations, equation dispatch) is inherited from HE unchanged.

The telescope walk checks arguments left-to-right: each successful type match extends the binding environment, and later domains and the codomain are instantiated under accumulated bindings. This is formalized in Lean (230 lines, 0 sorries) at `HEPrime/Telescope.lean`, with kernel-checked positive and negative examples:

- **Positive:** applying `dt` to `eqtl_rs1_gene1` infers return type `(target rs1 gene1)` via dependent substitution. Binary rule `ct` with cross-domain dependency (`$snp2` bound by first argument, constraining second domain) also kernel-checked.

- **Negative:** mismatched SNP (`rs2` $\neq$ `rs1`) correctly produces `none`. Standard HE telescope parsing treats the same domain as a plain atom, proving the semantics are distinct.

CeTTa implements HE′ as an opt-in profile (`he-prime`), gated by a single flag (`enable_dependent_telescope`). The implementation mirrors the Lean formalization: `split_dependent_domain`, `function_domain_type`, `bind_domain_binder`, and `infer_dependent_application_types` in `eval.c`.

10.1 Experimental: Typed Search over Dependent Codomains

The telescope elaboration above is the kernel-backed part of HE′: one interpretation function, mirrored in Lean. The `he-prime` profile additionally carries a larger *experimental* typing module (`src/he_typing.c`): consistency analysis, checked type-level computation, and proof search by type inhabitation. Its evidence is executable — a blessed adversarial regression suite (`tests/profile_he_prime_dtt_adversarial.metta`) and a reproduced end-to-end demonstration — and deliberately *not* a soundness theorem: no Lean correspondence exists for this module yet, and its final proof check reuses the same C implementation it validates. The two authority levels are distinct and should not be conflated.

The judgment. The module implements three-valued *success typing* over declared types. `check-typing` accepts a term against an expected type when *some* declared type is consistent with it, rejects when *every* declared type is a proven mismatch, and otherwise answers `unknown` with a reason (fuel exhaustion, ambiguous computation, inadmissible computation). A rejection is proven; an acceptance is not a promise. Consistency is licensed per edge by exactly one named reason — `exact`, `structural`, `dynamic` (`%Undefined%` on either side), `top` (`Atom` as the expected type), or `meta-staging` — and composite types combine children by *weakest license*: a single dynamic child keeps the whole edge dynamic at any nesting depth, so a value cannot be laundered from one base type to another through `%Undefined%` by wrapping it in a constructor.

Type-level computation. Dependent codomains may contain computation, so conversion is capability-gated at two levels with different strengths. A *direct* grounded operation reduces inside a type only if it lies on an explicit allowlist of deterministic, effect-free operations; any other direct grounded call — space mutation, I/O, a foreign call — is refused with `(he-reject (type-computation-inadmissible))` before dispatch and never executes. The typing operations' declared types stage their term and type arguments (`Atom` parameters), so argument evaluation cannot run an effect before the checker rules on it. A *user* function marked `type-level-function` is a weaker boundary: it evaluates against a fuel-bounded snapshot clone, so its *space* mutations are isolated and discarded, but external effects (for example I/O routed through the evaluator) are *not* capability-contained. That path is trusted and experimental; full effect exclusion would require evaluator-level effect capabilities or a separate pure type-reduction evaluator, not a marker plus a space clone.

Search as inhabitation. `type-inhabit-first` performs Curry–Howard backward chaining: rules are arrow-typed declarations marked `chaining-rule`, goals are types, and answers are terms returned together with a re-check of their type. Only proof-grade edges (exact or structural all the way down) admit a candidate: a witness whose type merely gradually matches the goal — through a dynamic or top license — can pass `check-typing` but never stands as an inhabitant. Forward chaining (`type-forward-closure`) reports `complete` only on a reached fixpoint, and otherwise `resource-incomplete` or `rounds-exhausted`, under one aggregate fuel budget.

Multi-answer enumeration. `type-inhabit` generalizes the existential search to a lossless multi-answer interface built on an explicit choice-point machine rather than eager recursive vectors. Premise proofs are pulled on demand from immutable resumable continuations, so distinct witnesses of one premise stay distinct top-level proofs — for a rule $A \to B \to C$ with two inhabitants of A, both proofs of C are returned — and the requested top-level answer count is never propagated into a child search. A ranked work queue schedules the finite depth-bounded candidates fairly, so a recursive rule listed before a direct one cannot starve the shallower finite proof. Answers are identified *losslessly*: two derivations are the same answer only when their canonicalized proof term, the substitution of the original query variables, and the re-checked normalized type all coincide, so proofs that differ only in a computed truth value or evidence term remain distinct. The interface is *prefix-preserving* — increasing the answer count or the fuel budget never removes or reorders an earlier answer; reaching the requested count with work still pending reports `more-possible`, and exhausting fuel returns the answers already found under `resource-incomplete`. This is an operational fairness guarantee over the bounded search space, not a general enumeration-completeness theorem: outside a certified-deterministic or completed-finite fragment the honest verdict remains `more-possible`.

Dependent-typed PLN, running. The payoff is the standard PLN idiom with truth-value formulas in rule codomains — the capability Geisweiller's DTL chainers had to leave disabled pending a type checker that runs functions inside types. Under `he-prime` the rules run as written (a derived encoding of the DTL constructor types, not an unchanged execution of the original source tree):

```
(: Deduction
   (-> (≐ $p $ptv) (≐ $q $qtv) (≐ $r $rtv)
       (≐ (→ $p $q) $pqtv)
       (≐ (→ $q $r) $qrtv)
       (≐ (→ $p $r) ; codomain COMPUTES the
          (deduction-formula $ptv $qtv $rtv $pqtv $qrtv)))) ; conclusion TV
(chaining-rule Deduction)

(: pP (≐ P (STV 0.8 0.9))) ; evidence facts (pQ, pR,
(: pPQ (≐ (→ P Q) (STV 0.75 0.8))) ; pQR analogous)

!(type-of &self (quote (Deduction pP pQ pR pPQ pQR)))
; => (type-set (≐ (→ P R) (STV 0.6333333333333334 0.7)))

!(type-inhabit-first &self (≐ (→ P R) (deduction-formula ...)) 2 500000)
; => (he-accept (: (Deduction pP pQ pR pPQ pQR)
; (≐ (→ P R) (STV 0.6333333333333334 0.7))))
```

The deduction truth value is computed *inside the type* during inference, and backward search returns the proof term with that computed type. Modus ponens and recursive evidence-counting rules run the same way. (The artifact writes the annotation constructor with the glyph U+225E, rendered ≐ here.)

Checked refinements. `validate-type-properties` runs declared per-index refinements: with a natural-index property declared for `VecN`, `(VecN String 2)` is accepted and `(VecN String -1)` is rejected with `(type-property-failed -1)` — an underflow that unrefined HE typing happily derives. The operation validates declared properties only; it is *not* a well-formedness judgment (no universes, no binder-domain formation, no positivity).

Open. A C$\leftrightarrow$Lean correspondence for this module; Σ/Id/ι formers and universe discipline (the module does dependent *elimination* plus checked refinements, not full dependent type theory); certificate generation; backward search over computed codomains whose truth values still contain free variables.

11 Generated Readers: the LanguageDef as Syntax Authority

A 2026-07 tranche (branch-local at the time of writing) replaces hand-maintained readers with parsers *generated from the authored grammars themselves.* Each language's surface syntax is written as a LanguageDef presentation (grammar productions, lexical scalar classes, and an atom-projection profile); a validating compiler normalizes the presentation into a parser intermediate form, classifies it into an admissible deterministic fragment, and emits native C readers linked into CeTTa. General GLL/GLR engines are retained as differential validators and as fallbacks for grammars outside the deterministic fragment.

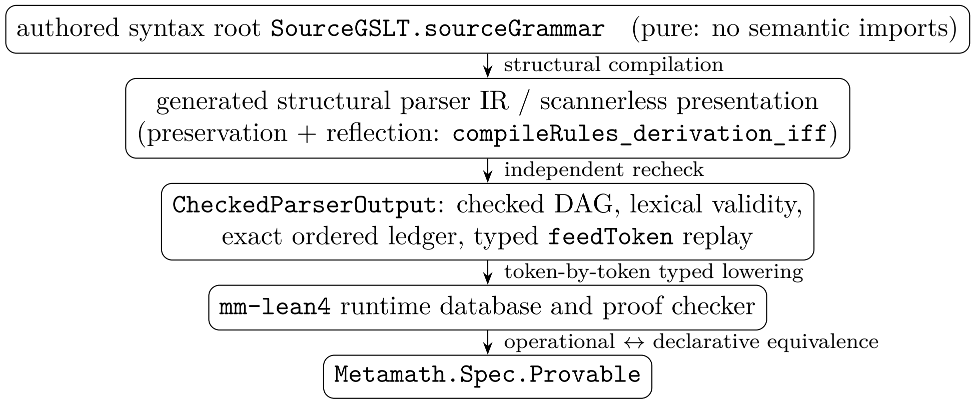
- **Per-language ownership.** The HE family, a two-stage PeTTa reader (document splitting, then per-form parsing — explicitly a syntax-only work-in-progress, not PeTTa evaluation semantics), and the Prime surface each compile from their *own* LanguageDefs into a shared projection-neutral runtime; no language is routed through another language's grammar.
- **The specification catches its own bugs.** The source-faithfulness classifier rejected two genuine defects *in the authored grammars* before execution: a token-start class that wrongly admitted `@` (making `@doc` both a quotation and an ordinary word), and a derivation ambiguity in which `*(e)` parsed both as a bare `*` followed by an expression and as one unquote atom. Both were repaired at the specification level — the relation was fixed, not the validator weakened — and re-verified in an independent Horn-clause model of the grammar.
- **Evidence.** Compiled-versus-legacy differentials (76/76 on the Prime surface, including Unicode, numeric projection, open-name rejection, and alpha-equivalent variable identity), compiler mutations killed, purity gates showing zero guest-language leakage across the generic engine and compiler files, and byte-identical regeneration of generated artifacts from their sources.

The architectural claim is deliberately bounded: this is certified *specialization* of admitted grammar schemata, with every generated component remaining an untrusted producer subject to differential and (ultimately) checker replay — not yet a general compiler from arbitrary grammars. Measured on the earlier prototype lane, a generated HE reader ran at 0.75×–0.87× the hand-written reader's time on standard-library parsing with exact canonical-tree agreement, indicating that grammar-as-authority and native performance are not in tension.

11.1 GSLT2Parse: syntax is a GSLT

The engine tranche above is the executable half of a formal programme we call *GSLT2Parse*, whose thesis is compositional: a language's *syntax* is itself a generated syntactic labelled transition system, and a parser is that syntax-GSLT's evaluator. One authored `LanguageDef` is therefore the single root from which the evaluator, the parser, and (through the Cost construction of Section 5.7) the resource semantics are all mechanically derived — three semantic artifacts, one authority, no hand-maintained agreement obligations between them. The formal side lives in the Mettapedia Lean development and is sorry- and axiom-clean across the roughly fifteen-thousand-line module family;

the axiom reports below were re-checked against the kernel at the time of writing. The pipeline, for the Metamath instance:



Parsing and checking remain separate judgments at every stage; the composition modules join them by theorem.

The generic compiler and its two-sided correspondence. A generic, language-agnostic compiler turns the grammar fragment of any `LanguageDef` into structural parser rules. Three theorems bound its authority. Changing any non-syntax field of the declaration cannot change the generated rules (`compileRules_eq_of_terms_eq`); generated-rule actions retain the independently decoded terminal/child skeleton, so a rule cannot silently drop, duplicate, or reorder children (`compileRules_action_shape`); and — the load-bearing result — derivability in the generated structural intermediate form is *equivalent* to derivability in the authored grammar at the ordered-token boundary (`compileRules_derivation_iff`: preservation *and* reflection, a genuine second relation rather than equality by definition; kernel axioms exactly `propext`, `Classical.choice`, `Quot.sound`; `GSLT/Parsing/LanguageDefSyntaxCorrespondence.lean`).

Checked languages: two laws, crowns for free. The reusable `CheckedLanguage` interface packages what any language must supply to close the loop from bytes to meaning: a syntax-only root, a lexical relation, a lowering into a checker, and a declarative judgment, subject to exactly two laws — exact lowering and checker adequacy. From those two laws the certificate-indexed and whole-pipeline crown theorems (`pipelineAccepts_iff_parserAccepts_and_declarative` and kin) are derived *once, generically*; the generic composition layer (`GSLT/CheckedLanguage.lean`) proves them with *no axioms at all.* Parsing and checking remain separate judgments throughout: the composition joins them by theorem, and neither the parser nor its certificate is ever allowed to authorize a theorem.

The Metamath vertical slice: a parser-to-provability crown. The first full instance runs from raw Metamath bytes to formal provability. The authored Metamath syntax root is a *pure* grammar — a semantic firewall: it imports only the generic grammar-inference layer, and the generic compiler contains no Metamath-specific names. On top of it, `CheckedParserOutput` demands an independently checked grammar DAG, lexical validity, exact ordered-token agreement, and a successfully constructed verified reader; `TypedLoweringCertificate` pairs every classified source token, in order and without insertion or omission, with an actual `ParserState.feedToken` transition of the verified `mm-lean4` checker, and proves that the instrumented execution erases to the production checker database. The crown (`generatedParserAndCheckerAcceptance_iff_specProvability`) then states that generated-parser-plus-checker acceptance is equivalent to declarative acceptance by `Metamath.Spec.Provable` — kernel axioms again exactly `propext`, `Classical.choice`, `Quot.sound`. The reverse direction cannot manufacture syntax evidence: it is indexed by an existing checked parser output whose certificates have already passed their independent checkers. The crown, verbatim from `SourceGSLTCheckerAlignment.lean`:

```
def GeneratedParserAndCheckerAccepts
    {bytes : ByteArray} (output : CheckedParserOutput bytes)
    (label : String) (formula : Metamath.Verify.Formula) : Prop :=
  TypedLoweringCertificate bytes output.source /\
    ImplementationAccepts bytes label formula

theorem generatedParserAndCheckerAcceptance_iff_specProvability
    {bytes : ByteArray} (output : CheckedParserOutput bytes)
    (label : String) (formula : Metamath.Verify.Formula) :
    GeneratedParserAndCheckerAccepts output label formula <->
      DeclarativeAccepts bytes formula
```

Executable closure evidence: seven checked ordered ledgers (demo0, MIU, Peano, disjoint-variable and proof-format cases, and an authentic propositional `set.mm` slice) in exact object-level agreement with independent `mm-lean4` output; normal/compressed proof agreement; five malformed-source rejections; killed mutations across labels, disjoint-variable conditions, compressed proofs, declaration order, AST, lexer, and semantic references; and byte-identical regeneration of both generated Metamath artifacts from the Lean root.

Second consumers and the engine loop. Megalodon has an honest second-consumer skeleton that reuses the generic signatures and crown — deliberately *not* yet claimed as an instance, since its verified checker and adequacy laws remain to be built. On the engine side, the generated Metamath grammar artifact is consumed directly by CeTTa (`lib_parse_metamath_grammar_generated_v0.metta`) alongside the `lib_parse` family and the signature-driven ABT binder layer in the Prime integration lane; the full engine-side parse gate (Metamath, ABT, and native-reader tests) was re-run green for this section. Honest boundaries: the tested `set.mm` artifact is a nontrivial authentic slice, not the full corpus; raw byte-to-token completeness is secured per accepted certificate by exact typed reader replay, with a universal raw-byte characterization theorem remaining future work.

12 MeTTa Prime: A Lazy Sibling Language

`-lang prime` is a *distinct* MeTTa language sharing CeTTa's implementation substrate: branch-local call-by-need with call-time choice as the ordinary application semantics. It is not an extension of HE and does not reduce to HE under erasure; the "HE unchanged" gates that accompany every Prime tranche establish noninterference on shared code, not a semantic relationship. Appendix C distills the living specification; the highlights are:

- **Need cells.** Suspensions are opaque runtime capabilities with an immutable origin and a checked cache state machine; completed values and stable faults memoize, blackholes and interruptions remain retryable. Call-time choice makes sharing observable: one delayed choice used twice yields only correlated outcomes.

- **Equation dispatch.** Rewrite candidates for one call share one source suspension per argument; the candidate *frontier* is a scheduling notion, never a kind of equality. The static frontier policies were all refuted as complete semantics by a common law audit; the selected direction keys producer identity to the Need cell itself (application and source positions are subscription edges), with a policy-neutral certified control layer under which breadth-first, depth-first, ranked, and permuted exploration provably preserve the answer bag.
- **Faults as data.** Ordinary evaluation propagates faults; explicit structural observers inspect a fault atom as data, so error-conditional programming is a deliberate boundary rather than an accident of evaluation order.
- **Anonymous variables.** Bare `$` is a fresh anonymous variable per occurrence (it matches any term and binds nothing referenceable); named variables retain co-reference. This is Prime-only: HE bare `$` remains governed by the HE written contract.
- **Dependence-aware evidence export.** A shared-cause discriminator is reproduced natively: two syntactically distinct explanations sharing one producer outcome yield exact possible-world probability 1/2 where naive independent-explanation disjunction gives 3/4. Prime exports dependence structure to evidence consumers; probability and proof authority never move inside the evaluator.

Early experiments host proof search as type inhabitation over the lazy frontier (Metamath-style problems with premise selection and reduction-ordering guidance), which is the intended cognitive-language role: demand-driven search over an inspectable frontier, with claim-bearing results lowered to an external checker rather than asserted by execution.

13 Related Work

Concurrent constraint programming. Saraswat et al. [2] established the theoretical framework for detecting stable properties of networks in concurrent logic programming languages. Their cc(↓,|) language, with Tell/Ask on shared variables over a monotonic constraint store, provides the semantic model that most closely matches HE/CeTTa's `match &self` evaluation. The distinction between monotonic constraint stores and independent-scope evaluation (Flat Concurrent Prolog) precisely characterizes the semantic boundary between HE and PeTTa.

MeTTa ecosystem. The MeTTaIL boot compilation pipeline [3] provides a contract-first, Lean-backed compilation path for MeTTa stdlib modules, complementing CeTTa's direct evaluation approach. The Mettapedia formalization project provides the Lean specifications (`EvalSpec`, `MeTTaEval`, `MatchSpec`) that ground both the translation proof and the CeTTa evaluator design. The PLN (Probabilistic Logic Networks) framework uses MeTTa's backward chaining extensively; the typed chainer pattern from Nil Geisweiller's `nilbc.metta` is the standard PLN idiom that works across all three runtimes.

14 Conclusion

CeTTa demonstrates that a direct C implementation of the MeTTa evaluator is competitive with PeTTa on inference workloads, but not uniformly faster. In the fresh reruns reported here, CeTTa wins on startup-dominated shallow inference and on several genomic-inference rows (the 183-pair

PLN evidence core and the 1,411,430-hypothesis deep genomic backward-chaining row), while PeTTa wins decisively on pure branching search (the 85,013-proof Metamath row). A cross-engine chaining suite (Section 6.8) confirms that CeTTa's shallow advantage is a startup-and-dispatch effect rather than a scaling property. Historical rows whose current drivers are unresolved or no longer the same computation as their old headline—the old miner speed table and BTree—are excluded from the revised benchmark ladder rather than carried forward on trust. Throughout, CeTTa maintains oracle-verified correctness across a 535-test MeTTa regression suite and a 23-case HE I/O semantic conformance gate against upstream HE v0.2.10. The sorry-free Lean proof of HE↔PeTTa semantic correspondence (5,400+ lines, 0 sorries), including import commutativity and operational bridges for state and space allocation, establishes that MeTTa's three runtimes are semantically compatible on the shared fragment. The compiled `.act` space format via MORK enables zero-copy indexed loading, demonstrated at the 100k-atom scale, with the large-scale attach path still under repair. On the process-calculus side, the RhoCalc reaction machine's eval-at-COMM semantics now has a witness-free, axiom-clean Lean characterization (Section 5.4): the quiescent run of the canonical payload-evaluation process is exactly its structurally observable outcome set, with no certification witness in the carrier.

Beyond raw performance, CeTTa now treats its own semantics as a first-class object. The profile family makes `he-compat` (literal Rust HE) and the principled `he` line distinct, recorded semantic products; the reflective modal surface exposes the one-step reduction relation—$\Diamond$, $\Box$, normal-form—to MeTTa programs themselves, aligned with the Lean OSLF derivation; and the small-step rule table makes the first reduction-rule classes named, reflectable, and removable, with rule-removal tests proving the engine depends on the declared rules rather than on duplicate hidden branches. The congruence bug that motivated this weld—an argument-congruence rule silently missing from the one-step relation on three branches at once—is exactly the drift class this architecture eliminates.

Tilepuzzle is settled capacity evidence again, but not a CeTTa speed win. The normalized contract counts 181,440 reachable boards including the start state; the PeTTa native example's 181,441 self-check is a counter-convention difference, not an extra board. The PeTTa native rerun remains much faster (1.40 s, 121 MiB), while the CeTTa result is the repaired capacity row: the full board-count search now completes in 8.80 s at 233 MiB after the tail-safe evaluation-arena garbage collector (Section 7.2). Trail-style rollback, bindings builders, and arena mark/reset remain candidates for further improving search-heavy workloads without converting CeTTa into a Prolog engine. The guiding principle: *backtracking should be rare, local, and hidden—not global, ambient, or user-visible.*

Future work.

- **Complete HE one-step coverage:** equation matching is table-routed; migrate the special forms (`chain`, `let`, `let*`, `case`, `switch`) one rule class at a time, each with bag-equality and rule-removal gates.
- **`-compile-langdef` codegen:** a generator emitting both the reflectable pack and specialized C for the hot evaluator under one shared source digest, extending the `-compile-stdlib` blob precedent; an evaluate-by-iterating-one-step mode as reference oracle.
- **Small-step / official-HE correspondence:** the `HESmallStep` relation and its OSLF span exist in Lean (proven; zero sorries); the remaining climb is the delimited collapse/simulation theorems connecting it to the fine `mettaHE` instruction machine.

- **Bounded modal operators:** fuel-bounded `reachable-within` ($\Diamond^{\leq k}$) and `invariant-within` ($\Box^{\leq k}$) over the now-faithful one-step relation.

- **Local search machine:** trail/watermark rollback, `BindingsBuilder`, arena mark/reset for scratch atoms. Target: materially improve search-heavy workloads such as tilepuzzle; the state-count contract is settled and the earlier arena-growth regression is closed by the evaluation-arena garbage collector, so this is now an optimization opportunity rather than a correctness or capacity blocker.

- **Indexed shared-space engine:** leapfrog triejoin (from Vandervorst's `trie_synth.py`), conjunction query planning, SpaceEngine boundary (formalized in `SpaceEngineBoundary.lean`: native ⊂ pathmap ⊂ mork, with ACT as artifact-only surface; `ACTArtifactBoundary.lean` classifies `open-act/load-act!/dump!` as storage seams). Target: 10–100× on large-KB multi-pattern conjunctions.

- **MM2 as MeTTa data (working):** MM2 exec rules are atoms in MORK-backed spaces. MeTTa creates, inspects, rewrites, and steps them through the ordinary space API. `step!` is the execution interface; `.mm2` files import via `include/import!`. MeTTa can pattern-match over MM2 code to analyze or transform programs—the meta-language relationship is native.

- **PathMap algebra (working):** CeTTa already exposes MORK's PathMap substrate through the `mork:` surface: the lattice operations `mork:join`, `mork:meet`, `mork:subtract`, and `mork:prefix-restrict`; read-only zipper navigation; product-zipper Cartesian joins over two to four spaces; and overlay-zipper cursors. Underneath, the Rust substrate is substantial—zipper navigation (5,800 lines), product-zipper joins (2,000 lines), and the lattice operations (1,600 lines)—and a Lean formalization (46 files, 19K+ lines) proves the three-phase execution protocol, fold-level aggregation, and PathMap lattice correspondence. What remains future is surfacing MORK's aggregating sink types (count, sum, head, hash, and, Z3, WASM) and source types (BTM, ACT, equality/inequality constraints) at the MeTTa level: the kernel implements them, but the `mork:` API does not yet expose them.

- **HO search engine:** Lash-inspired SAT-driven proof search for PLN and type-directed synthesis. PicoSAT as embedded SAT solver. Separate from the evaluator.

- **Translator CLI:** `translate.sh` with `he2petta/petta2he`, recursive and bundle modes. Lean proof covers pure fragment + state + spaces (0 sorries).

- **Reaction-semantics formalization:** extend the witness-free eval-at-COMM characterization (Section 5.4) from the single-reaction bag fragment to concurrent and persistent reactions, and relax the hash-set-free channel side condition.

- Full pattern miner on CeTTa end-to-end (∼12× speedup over PeTTa).

A LeaTTa Cross-Check Appendix

LeaTTa is a Lean 4 MeTTa runtime and proof development that is useful as an independent semantic cross-check for the HE-facing core of CeTTa. The local checkout used for the measurements in this appendix was a clean LeaTTa 1.0.6 tree at `github.com/MesTTo/LeaTTa`, commit `51b4b52` (`2026-06-22`, "prepare 1.0.6 release"). Upstream release notes and the checked-claims book are published at `mestto.github.io/LeaTTa/`.

This appendix records a local scope-and-spot-check run on 2026-06-24; it is a coverage map, not a new benchmark ladder replacing the main paper. The CeTTa side used `-profile he-extended -lang he`. The LeaTTa side used `-oracle` for assertion-bearing files and `-file` for pure loaders. A bulk pass ran every `.metta` file under the CeTTa tree (1255 files) through LeaTTa.

Coverage, in three tiers. The raw aggregate—168 of 951 assertion-bearing files passing in full—is misleading on its own, because most of the CeTTa test corpus exercises extension surfaces that LeaTTa, a deliberately minimal HE interpreter, does not implement and is not meant to. The honest decomposition is:

Tier	What LeaTTa does on it
HE core	**Strong.** Of the 21 `he_*` conformance files, 18 pass in full and 2 partially (one loader carries no assertions), for 203/206 assertions. Full passes span symbols, equal-chaining, backchaining, GADTs, dependent types, higher-order functions, grounded basics, type propagation, states, documentation, and PLN truth values. This is the surface that overlaps the HE evaluation semantics.
CeTTa extensions	**Out of scope by design.** The bulk of the 783 partial failures call builtins LeaTTa does not provide: MORK-backed spaces (84 files), PathMap (44), `hyperpose` (27), typed `new-space hash/queue` (12), `import!` (10), `space-set-match-backend`, and the textual Metamath parser. Failing these reflects LeaTTa's minimalism, not a fidelity gap.
Genuine gaps	**In-scope divergences.** A float-only numeric model: `(min-atom (5 4 6))` returns `4.000000` rather than an integer, and rationals such as `1/2` raise "only numbers allowed," so the exact integer/rational tower does not correspond. Typed error contours (e.g. `(+ 5 "S")` → `BadArgType`, empty/wrong-arity `car-atom/decons-atom/min-atom`) are not reproduced; we treat these as accepted divergences. Two `he_b4` assertions touching inert-symbol and empty-match semantics, and the higher-order WM-PLN bridge (218/221 on this checkout), remain to be diagnosed.

Spec repair triggered by the cross-check. The LeaTTa comparison was useful not only as an oracle but as a debugger for our own HE Lean surface. During this pass we found that the public equation-query path had been routed through the simplified `simpleMatch` helper even though the faithful two-sided `matchAtoms/mergeBindings` semantics already existed. The repair is now written down in three Lean files: `DeclMatchSpec.lean` gives a declarative specification of `match_atoms` and proves `matchAtoms_sound/_complete`; `DeclMergeSpec.lean` does the same for `merge_bindings`, proving `mergeBindings_sound/_complete`; and `QuerySurfaceAgreement.lean` proves the staged ground-fragment adequacy theorems `queryEquations_ground_two_way_adequate` and `queryEquationsAgainstVisible_ground_two_way_adequate`. So the cross-check is now anchored on the faithful ground-query surface rather than on the older simplified one. The remaining variable-bearing query case is deliberately left explicit: it requires the next repair, namely threading equalities through `Bindings.resolve/Bindings.apply`, and is the right next theorem rather than a hidden assumption.

Performance. On matched runs LeaTTa is correct where it is in scope but slower and markedly more memory-hungry than CeTTa's C runtime—operational overhead expected of an executable Lean interpreter, not a semantic limitation.

Benchmark	CeTTa	LeaTTa	Note
tail_recursion_deep_regression	0.30 s	0.80 s	both pass 3/3 assertions
tail_recursion_deep	0.30 s	0.80 s	both pass 3/3 assertions
he_c3_pln_stv	0.10 s	0.10 s	both pass 5/5 assertions
he_d2_higherfunc	0.10 s	0.10 s	both pass 25/25 assertions
test_wmpln_ho_similarity_bridge	0.10 s	0.10 s	CeTTa passes; LeaTTa passes 218/221 assertions
bio_refseq_723k	0.40 s	3.30 s	both run; LeaTTa ≈1.16 GB RSS vs. CeTTa ≈196 MB
bio_tfbs_415k	0.30 s	2.10 s	both run; LeaTTa ≈917 MB RSS vs. CeTTa ≈136 MB

Tilepuzzle: surface vs. performance. The raw CeTTa tilepuzzle benchmark depends on CeTTa's typed `queue` and `hash` spaces, so it does not run unchanged on LeaTTa (a Tier-2 surface mismatch). A bounded adaptation replacing those spaces with a functional queue and plain named-space membership does run: a 100-step witness reaches 171 visited states in 18.47 s, while the 500-step version did not finish under a 30 s cap. For reference, the clarified native contract counts 181,440 reachable states including the start board, while the older PeTTa example's 181,441 self-check is a counter-convention difference, not evidence for an extra reachable board. A fresh full native CeTTa run now completes with the tail-safe evaluation-arena garbage collector: 181,440 states in 8.80 s at 233 MiB (Section 7.2). So this bounded LeaTTa adaptation is only evidence that the surface idea transfers, while the full native CeTTa row is settled capacity evidence.

Bottom line. LeaTTa is a credible independent oracle and a candidate verified engine-floor for the HE evaluation core of CeTTa: it agrees on the conformance surface that matters, diverges only on extension builtins it deliberately omits and on a float-only numeric model, and pays an expected interpreter-speed cost. Linking LeaTTa to the HE Lean specification is therefore well-founded on the core, provided the numeric tower and error-contour divergences are encoded as known boundaries. The specification side is now in a better state than when this appendix was first drafted: the ground-query fragment is connected to the faithful HE matcher/merge semantics by proved two-way adequacy, and the remaining variable-bearing query case has been isolated cleanly enough to be attacked next rather than smudged into the oracle story.

B Experimental — MAM: The MeTTa Abstract Machine

This appendix is explicitly experimental. It captures design exploration from ongoing discussions (April 2026) between the authors and external reviewers (GPT-5.4 Pro), concerning the post-phase-1f abstract-machine layer for CeTTa. *Nothing here is yet implemented or committed architecture. It is recorded to preserve design rationale; the authoritative implementation plan remains* `implementation_plan.tex`.

B.1 Motivation

The phase-1f-era extraction seam (`src/search_machine.{h,c}`) pulls policy parsing, stream combinators, and a `TableStore` plugin out of `eval.c`, and wraps them in a thin callback struct `CettaSearchMachine = {stream_source, eval_bound_single, eval_ctx, table_plugin}`, stack-allocated at each call site. This is a useful refactor, but it is a *plug interface* — not an abstract machine in the WAM / ZAM / SLG-WAM sense. Real abstract machines (XSB's

SLG-WAM, SWI's tabling runtime, Soufflé's compiled Datalog, miniKanren's CPS-transformed search) have persistent machine state, explicit visitor protocols, capability advertisement, and episode lifetimes.

The proposal here is a *principled* next layer that retains the current code as `SearchAdapter` (transitional helper) while naming the real abstraction from first principles.

B.2 Current categorical reading

The MAM's present semantic reading, under active revision, models it as a **quantale-enriched traced profunctor**, fibrated for reflection. This reading is provisional — it captures the current best understanding of the structure, not a definitional claim that binds future architecture. Unpacked:

- **Profunctor** $P : \mathcal{Q}^{\mathrm{op}} \times \mathcal{A} \to \mathcal{V}$: *search.* Queries in $\mathcal{Q}$ relate to answer streams in $\mathcal{A}$, with intensity weighted by elements of $\mathcal{V}$.

- **Quantale enrichment** $\mathcal{V} = (L, \otimes, \leq, \bigvee)$: *multiplicity.* Answer weights live in a quantale: $\mathcal{V} = \mathbf{2}$ for classical boolean search; $\mathcal{V} = \mathbb{N}$ for counted answers; $\mathcal{V} = [0,1]^2$ (strength $\times$ confidence) for PLN. PathMap's trie-algebra is natively a quantale.

- **Trace** $\mathrm{Tr}_X^{A,B} : \mathrm{Hom}(A \otimes X, B \otimes X) \to \mathrm{Hom}(A, B)$: *fixpoint.* Feedback / saturation / tabling / cyclic queries are all instances of categorical trace (Joyal–Street–Verity, Hasegawa).

- **Kleisli structure over an effect monad** (or algebraic effect handlers *à la* Plotkin–Pretnar): *control.* Demand, cut, suspension, resumption, and schedule are composable effect operations.

- **Fibration** $p : \mathcal{T} \to \mathcal{B}$ (optional, for reflective MeTTa): rules-over-machine-state.

A trace operator is provably equivalent to taking the least fixpoint of a parameterised endofunction (Hasegawa 1997); this is why tabling, PLN-convergence, and saturation all fit one uniform operator.

B.3 The proposed machine family: MAM

We name the machine family **MAM: MeTTa Abstract Machine**. The name is chosen by identity rather than by current structure or scope: MAM is the abstract machine for the MeTTa language family, and that identification remains accurate as the internal architecture evolves. The naming tradition (WAM, ZAM, STG, CEK, CAM) uses either author identity or component decomposition; MAM uses the language identity it serves, which is stable across architectural iteration. Specifically, MAM does *not* hardcode a categorical claim (as "QAM = Quantale Abstract Machine" would have done), and does *not* hardcode a functional scope (as "SCAM = Search & Control Abstract Machine" would have done). The categorical reading and the current feature set are both free to evolve without renaming.

Concept	**Name**
Machine family	`MAM` (Quantale Abstract Machine)
Formal description	Quantale-enriched traced profunctor
Engine 1 (goal-directed, Kleisli/effect)	`Chainer`
first realization (native)	`native_chainer`
Engine 2 (fixpoint, trace)	`Saturator`
first realization (pathmap)	`pathmap_saturator`
Per-session state	`Episode`
Dispatch vtable	`MAMOps`
Answer	`Answer` / `WeightedAnswer`
Schedule axis	`dfs` / `bfs` / `iddfs` / `interleave` / `native`
Demand axis	`unbounded` / `once` / `limit-k` / `fold`
Visitor control	`CONTINUE` / `STOP` / `ERROR` / `SUSPEND`

B.4 The two-engine pressure strategy

Rather than over-specifying a first concrete machine, MAM is pressured from day 1 by *two* independent realizations that stress complementary axes of the seam:

`Chainer` (goal-directed). WAM-lineage control: explicit environments/frames, choicepoints, trail/undo, deterministic tail-path handling, bounded-demand response, direct conjunction when advertised. Design-guide workload: **metamath backward chaining**. Before the evaluation-arena garbage collector this workload could exhaust memory on CeTTa; with the collector CeTTa now completes the Metamath proof-search rows (Section 6), so the remaining motivation here is WAM-lineage control, not a memory-exhaustion gap. First realization: `native_chainer`.

`Saturator` (fixpoint). SLG / Datalog / saturation lineage: snapshots, delta-sets, conjunction planning, exact membership, counted answers, eventual path toward PLN and ATP-style saturation. Design-guide workload: **PLN multi-pattern deduction**. First realization: `pathmap_saturator` (because PathMap's quantale structure makes weighted answer aggregation natural).

This pair creates the right kind of pressure on the seam: Chainer forces frames, choicepoints, `EnvRef`, demand handling, and explicit scheduling; Saturator forces snapshots, delta sets, conjunction planning, exact membership, counted answers, and the eventual path to PLN and ATP.

Compiled / bytecode (ZAM-lineage) and ATP saturation engines are explicit *later* lanes — they risk freezing the protocol in the wrong shape if pursued before Chainer and Saturator coexist cleanly under one MAM.

B.5 Axes that must stay separate

A core architectural commitment is that the following axes are **distinct data** in `SearchRequest`, not tangled behind surface-level operators:

Schedule. `dfs` / `bfs` / `iddfs` /`interleave` are *fairness policies*, not engines. Korf's classic result motivates `iddfs` as a completeness-friendly default over raw `bfs` (BFS completeness with DFS memory); miniKanren's interleaving exists precisely to avoid starvation on infinite branches.

Demand. `once` / `limit-k` / `unbounded` /`fold` describes how many answers are requested, independently of schedule. `once` and `select 1` must build `demand = FIRST`; they must *not* silently hardcode DFS.

Table mode. `none` / `variant` / `subsumptive` /`incremental` mirrors the XSB / SWI ladder; each is an explicit engine-level choice.

Backend. `native` / PathMap / MORK chosen by the actual `Space`, not by a global preference.

Lane. `recursive-dependent-proof` / `atp-saturation` /`solver-oracle` / ... chosen at surface.

The request builder (one per `-lang`) normalizes surface syntax into `SearchRequest + SearchLangContract`; the MAM consumes the normalized object, never raw surface atoms.

B.6 The seven-type vocabulary

Seven types are proposed to land together before the first real backend optimization (counted-answer variant table, PathMap-native conjunction, native exact-match index). Landing them simultaneously prevents the "first optimization forces a rewrite" pathology.

Type	Categorical role
`SearchRequest`	Object of $\mathcal{Q}$
`SearchAnswer`	Object of $\mathcal{A}$ (quantale-weighted)
`SearchLangContract`	Subcategory inclusion $\mathcal{C} \hookrightarrow$ MAM
`SearchCaps`	Capabilities = presence of morphisms/natural transformations
`VisitCtl`	Algebra of the effect operation visit
`MAMOps`	Kleisli dispatch vtable
`Episode`	One trace-iteration instance

Implementation-actual `SUSPEND/RESUME` behavior, cancellation tokens, snapshot-token machinery, and per-`-lang` registration remain explicitly deferred.

B.7 Reference base

The proposal draws on the following literature, all available in the local library:

- *Aït-Kaci 1991, WAM Tutorial Reconstruction* – foundational WAM semantics; drives `Chainer` design.
- *Swift & Warren, XSB System Manual; Sagonas et al. 1994* – SLG-WAM tabling; drives `Saturator` / table-mode ladder.
- *Scholz, Jordan, Subotić, Westmann 2016, Soufflé: On Fast Large-Scale Program Analysis in Datalog (CC)* – staged compilation of fixpoint computation; compiled `Saturator` path.
- *Keoliya & Byrd 2020, microKanren* – interleaving search as a distinct schedule axis.
- *Ager, Biernacki, Danvy, Midtgaard 2003, A Functional Correspondence Between Evaluators and Abstract Machines (BRICS)* – derivation methodology for machines from evaluator semantics.

- *Leroy 1990, The ZINC Experiment* – machine-description vs. run-instance separation (maps to `MAMOps` / `MAM` / `Episode`).
- *Russell & Norvig, AIMA Ch. 3* – IDDFS completeness; uniform treatment of uninformed search.

B.8 Status

Since this appendix was first drafted, the derivation methodology it cites has acquired kernel-checked foundations in the Mettapedia Lean tree (`Mettapedia/Machines/`): a functional-correspondence module deriving a CEK-shaped eager machine and a Krivine-shaped demand-driven machine from fueled evaluators over one shared term substrate, with the evaluator-to-machine correspondence and machine determinism proved; a cone-duality module (forward/production and backward/demand reachability cones, their Galois adjunctions, wavefront exhaustion, and the demand-diamond restriction theorem); and a substrate module packaging the two machines as control cores over one term type with a kernel-checked strategy-separation witness (a single term on which the eager core returns nothing at every fuel while the demand core answers). Axioms for every keystone are within {propext, Classical.choice, Quot.sound}. These results turn "the machine is derived from the semantics" from a methodological citation into project theorems: the eager and lazy language lines require distinct control cores over a shared substrate, and demand-restricted evaluation is sound and complete for goal-reaching by theorem rather than folklore.

This appendix otherwise records design rationale; it does not commit implementation. Reasoned design review converged on ∼0.88 confidence on MAM naming and two-engine strategy. The concrete implementation plan (phase 2 seam; phase 3 tabling plugin; phase 4 backend parity) is the authoritative schedule and is unchanged by this appendix.

This text should be revisited at each phase boundary and revised as implementation reveals which proposals held and which did not.

C Appendix: The Prime Living Specification, Distilled

This appendix distills the Prime living specification and decision ledger as of 2026-07-23. The living document separates selected design, implemented behavior, verified evidence, proposals, and open questions; every consequential clause carries a counts-primal evidence annotation (independent supporting items n^+ and unresolved counterexamples n^- are primal; a strength/confidence pair is derived from them, and retired refutations are kept on the record rather than deleted).

C.1 Constitutional boundary

Prime is a distinct language: branch-local call-by-need with call-time choice is its ordinary application semantics, while the HE-family profiles keep their existing evaluation semantics; shared backend refactoring is permitted, silent semantic bleed is not. Execution produces answers, effects, coverage, and receipts — never theoremhood; claim-bearing results lower to an external generic checker for replay.

C.2 Design principles (P0–P11)

P0 (meta). Prime aims to be as good as theoretically possible as a cognitive language: specification details must follow from best-possible elegance, ergonomics, performance, and coherence

— and from the principles below — rather than from incident, imitation, or implementation convenience. A clause that derives from no principle is either evidence of a missing principle or a candidate for removal.

P1. Checking-first authority: engines, machines, indexes, and learned guidance are producers or schedules; a generic checker replays claim-bearing derivations. A hit is not a proof.

P2. Canonical meaning is distinct from execution schedule; an optimized schedule must refine the declared relation.

P3. Identity is semantic when sharing is observable: suspension, answer-occurrence, branch, effect, and capability identities are not interchangeable, and serialization may not erase aliasing or correlation.

P4. Choice preserves occurrences and correlation: results form an occurrence bag; one call-time suspension is shared by every occurrence denoting the same source argument; explicit resampling creates fresh choice identity.

P5. Effects are never hidden by backtracking vocabulary; sibling branches do not share mutable state by default; commit and merge are explicit policies with receipts.

P6. Completion is data: *incomplete* is not *false*; query polarity determines what one witness or one counterexample can settle.

P7. Scope and names are explicit layers: canonical binding is signature-driven de Bruijn; matcher metavariables are a separate layer; quoted structural names present identity without conferring resolution.

P8. Quotation does not grant authority: quoting makes syntax inert and nameable; privacy, liveness, reachability, and authorization are resolver and capability properties.

P9. Unknown syntax remains data: a form is evaluated only when an admitted operation, equation, or judgment determines how; otherwise it is residual, and an error requires positive evidence of a violated contract.

P10. Open extension is package-indexed, with formation rules, capabilities, and correspondence status; dynamic changes invalidate caches by digest.

P11. Theoretical analyzability is an implementation requirement: state-changing operations correspond to named transitions; tests carry positive, negative, and mutation witnesses per semantic boundary.

C.3 The Need core

A cell separates an immutable origin (term, lexical environment, package revision, provenance) from a checked mutable cache (`Empty` → `Evaluating(owner, generation)` → cached value or stable fault), with same-owner recursive demand reported as a blackhole (receipted, not cached, cell restored) and other-owner demand routed to an explicit wait/help protocol (selection open). Only faults stable under re-execution in the same world are cached; resource exhaustion and cancellation retain a resumable frontier. `delay`, `force`, and `resample` are the surface (lower-case only); `resample` deliberately breaks call-time correlation.

C.4 Ratified in the 2026-07 sprint

- Bare `$` as a fresh anonymous variable per occurrence (parser 25/25, reference model 29/29, grammar tournament 16/16, evaluator/quotation matrix 99/99 with sanitizers, a mechanized unique-selection theorem, and a corpus check showing no existing program depended on the old reading); the rejected root-binder alternative is retired on the record.
- Producer identity is the Need cell (store-lineage allocation identity), not application or source position — a native alias probe shows one shared cell observed from four application sites remaining correlated; the four static frontier policies fail a common fifteen-law audit at 9, 10, 11, and 12 laws respectively.
- A policy-neutral certified control layer: controllers reorder certified frontier nodes and can neither create nor destroy answers; pruning is explicit. Exhaustive policy invariance is proved; finite-budget fairness is a recorded obligation, not assumed.
- The authored Prime LanguageDef as executable syntax authority through the generated reader (Section 11).
- An equality vocabulary pin: directional rewriting, computed equality, unreduced syntactic comparison, alpha-equivalence, and registered/conversion equality are five different relations, and the candidate frontier is scheduling, not equality.

C.5 Open

The public suspension-observation surface (constraints and invariants selected; the concrete observed form is a narrowed choice); the worlds carrier presentation (event structures lead); registered-equality surface syntax; the other-owner forcing protocol; cyclic graph persistence; typed-laziness coherence as a mechanized refinement; the cell-causal native contender behind its confidence gate; and finite-budget fairness under adversarial ranking.

D Appendix: Core Design, Implementation, and Specification Principles

Distilled from the project's design discussions (June–July 2026) and the implementation campaigns they governed. Principles are grouped; each group opens with its load-bearing head. They overlap deliberately with Appendix C: the Prime principles are the language-level refinement of these project-wide commitments.

B.1 Authority: who may mint truth

Checking-first. Producers — search procedures, elaborators, learned models, external provers, the runtime itself — are untrusted proposers; theoremhood is conferred only when a generic checker replays an explicit certificate against an admitted presentation. Corollaries: authority flows through replayable proof objects, never execution traces of another engine; certificates carry their substitutions (the checker applies and compares, never unifies) and name their exact subject; a capability is enabled only by a machine-checked certificate of the property that makes it safe (checker conversion requires a confluence/canonical-section certificate; deterministic parser specialization requires a determinism proof); passing test suites are evidence, never theorems; and checking is search with the answers pinned — one kernel, two guidance policies, heuristics never deciding truth.

B.2 Inertness and honest verdicts

Unknown stays inert. The evaluator does not evaluate what it cannot interpret: unmatched forms are their own normal form, and no profile emits "feature unavailable." Externally this becomes a layered vocabulary: unknown syntax is residual; a recognized protocol with an unresolved endpoint is an explicit pending value; transport failure is a fault receipt; service rejection is a rejection value. Judgments return proven, rejected-with-reason, unknown, or resource-incomplete — unknown is never rejection, exhaustion is never quiescence; no verdict may flip as fuel increases; "must" judgments require nonemptiness plus completeness; and fuel is never a semantic premise — the semantics is pure, with resource outcomes layered outside it.

B.3 Specification as source

Syntax is a GSLT; one root per language. Concrete syntax is itself a graph-structured lambda theory — streams as terms, productions as rewrites — and the parser is that theory's evaluator (print-parse as a section–retraction law; unambiguity as a confluence theorem). Each hosted language has exactly one mathematical root; the engine supplies generic mechanism only. The genericity gate: hosting language $N+1$ must be a zero-line diff in generic code — only new data. Roots change by governed correction (impossibility witness plus sign-off), never by silent drift; hand-authored artifacts styled as generated are forbidden. Generated artifacts pass falsifier ladders (acceptance, AST differentials against an independent reference, near-miss rejection, print-parse stabilization, per-production coverage), and failures repair the declarative specification, never the engine. The specification itself is self-hosted, queryable data — properties, decisions, and support edges with explicit status — in which refuted claims persist as refuted nodes so they cannot silently resurrect. Generic machinery is verified against stable semantic authorities first; a rapidly evolving language rides as a canary, not an anchor. Before accepting an obstruction, ask whether it survives in the canonical mathematical object or is an artifact of the encoding.

B.4 Minimality

Mechanism in the kernel, policy in the library. The core owns rewriting, the checking judgment, and one resumable fair scheduling primitive; strategies — including proof search as type inhabitation — are compiled library code, inspectable and overridable. Match, unify, communication, comprehension, and inhabitant selection are one primitive. Inference rules are semantic; direction, agenda, indexing, tabling, and saturation are schedules over the same rules. A candidate core construct must serve at least two of practical programming, native dependent typing, and guest-logic adequacy, or it is an extension package; each core principle carries an independence witness (something breaks without it, nothing else forces it).

B.5 Pluralism

Profiles are conservative extensions; no sovereign object theory. A profile is legitimate while existing programs run unchanged; the first non-gateable conservativity violation is the dialect trigger, and minting a dialect is a normal event — the language is a family. The generic waist standardizes how rules are replayed, never what guest judgments mean; at least two object logics are maintained at all times so no package is both the definition and the validation target. "The checker accepted a derivation relative to package P" is categorically different from "P is sound"; bridges between logics are theorems or they are nothing.

B.6 Homoiconicity, reflection, binding

No second grammar, and quotation confers no authority. Types, judgments, packages, and claims are ordinary atoms the reasoner can inspect and rewrite; one sigil-to-expansion table drives the reader and both printers, so compact and expanded notation are one language. Binding is "both, by kind": matcher metavariables keep identifier machinery while object binders are canonically de Bruijn, meeting at elaboration, printing, and replay seams; stored checked terms are closed and alpha-equality is structural equality. Reflective observation is taxed at the dangerous act, not the epistemic one: looking is cheap and pure, forcing carries the ceremony, and receipts attach where claims are made.

B.7 Typing and graduality

Graduality is a same-program theorem. A validly typed working program with annotations erased computes the same results — always typed-versus-untyped of one program, never a cross-profile comparison. Semantic value types, syntactic shapes, and staging are three vocabularies; the gradual dynamic type has deliberately non-transitive consistency (no laundering a type through "?"); permissive success typing is never advertised as sound gradual typing without cast-and-blame enforcement; and there is no subtyping without coercion coherence — honest poverty over dishonest wealth.

B.8 Equality

Stratified, weakest-in-core. Identity and typed equality answer different questions (a cross-type comparison is ill-posed, not false); commitments form a lattice (same-atom, conversion, equal-at-type, identity, observational equivalence) with the core adopting the weakest adequate one. Program equations are directional rewrites; checked conversion is admitted only from certified fragments; view equations are pure projections; semantic equivalences are separately proven — a directional rule never silently becomes checker-authoritative symmetric conversion, and no registered equality may identify a suspension with its origin or two cells by origin equality, because observation is a projection and identifying across it destroys sharing.

B.9 Laziness, sharing, capability

Sharing is semantics. Branch-local call-by-need with call-time choice makes cell identity observable: a shared argument forces once per branch and every consumer sees the same value. A suspension has three strictly ordered observables — cell identity, origin, eventual result — and the strictness *is* call-time choice. Origin is an allocation-time snapshot, so optimization cannot change what inspection sees; observing never forces and never leaks the cell capability; re-delaying an observed origin allocates fresh. "May force" and "may inspect" are separate authorities; names are addresses while capabilities are authority (knowing a descriptor authorizes nothing); foreign services are one typed request/reply protocol with receipts, one suspension being one request with a memoized response and resampling the explicit freshness boundary — which also gives sample-once stochastic choice for free.

B.10 Occurrences, cost, and enrichment

The non-conflation chain. Legal transition $\neq$ occurrence identity $\neq$ probability/rate $\neq$ scheduling priority $\neq$ resource cost $\neq$ utility $\neq$ provenance/authority; every enrichment is a readout of one relational process. Results are occurrence bags with completion status; answer identity is defined before any deduplication; rates attach to occurrences, never branch counts, so refactoring-invariance of the measure is a theorem obligation. Cost receipts are measured pomsets whose algebraic laws are derived, never posited — and when a desired law met a kernel-proved obstruction it was withdrawn by counterexample and repaired by indexing, not weakened by assumption. One law appears three times: erasure recovers the base (gradual typing's erasure theorem, profile noninterference, cost observer-transparency) — "the definition of 'this is still MeTTa'." Control fields route compute, never truth; every legal transition keeps a protected baseline rate (no starvation, anytime behavior); reversibility is an effect property, not a global mode; and a system that modifies its own laws accepts a delta only with a proof that the delta preserves what it claims.

B.11 Effects, errors, resources

Unmetered by default; errors are values. Budgets and ledgers are opt-in and zero-cost when absent; effectful power is capability-gated and never hidden inside conversion; errors are inspectable, matchable atoms in the result bag — branch-local, never process-killing, and richer than a boolean; default judgment forms carry no fuel argument, and exhaustion surfaces only when it changes the answer.

B.12 Evidence and confidence

Counts-primal, ladder-honest. Hard pruning requires proven legality or impossibility with recall preservation; everything uncertain is soft ranking, demonstrated by a counterexample of an uncertified mask eating a real solution. Evidence accumulates as counts with explicit independence accounting (correlated descendants never count twice); the confidence ladder distinguishes kernel-factive "program meets specification" from the graded "specification captures intent," whose independent review is the second source; specification authors are kept blind to candidate artifacts; and a reviewed specification plus a kernel-checked divergent witness is a factive, *generative* refutation — first mismatch, certified continuations, repair targets.

B.13 From semantics to machines; method and naming

Machines are derived, not designed. Abstract machines arise from evaluators by the functional correspondence, so the eager and lazy lines share a substrate while requiring distinct control cores — now a mechanized fact — and demand-driven execution restricted to the production-cone/demand-cone diamond is sound and complete for goal-reaching, making magic-set restriction, premise pruning, and tabling instances of a theorem. Method: contested surface laws are settled by feature-ablation tournaments (every candidate behind a flag, one corpus, discriminating programs choose); falsification ladders run theory-decidable gates before compute is spent ("no conservativity, no experiment"); independently written engines share defect classes, so the executable-independent declarative specification is the productive referee. Naming: mathematical vocabulary is used only after its preconditions are theorems ("earn the word"); the algebra is the specification, stated in interface headers and enforced as gates; one name, one meaning, no aliases after renames; anti-goals name integrity violations only, with scope stated positively; ergonomics is a first-class late phase, redone against collected real confusion once the semantics settles.

B.14 Supersession, on the record

The principles themselves are governed by B.3: superseded forms stay recorded rather than deleted. Notable retirements: notation as a side-record superseded by syntax-as-GSLT; early list-shaped cost receipts superseded by measured pomsets, whose desired law was withdrawn by kernel counterexample and repaired by parameterized-monad indexing; blanket trace-refusal refined to generator-based replay of standard certificate formats; "freeze" vocabulary replaced by governed change with impossibility witnesses; and a draft rule promoting every program equation to checker conversion refused in favor of the stratification of B.8.

References

[1] B. Goertzel et al., "MeTTa: A Reflective Language for AGI," OpenCog Foundation, 2024.

[2] V. Saraswat, D. Weinbaum, K. Kahn, and E. Shapiro, "Detecting Stable Properties of Networks in Concurrent Logic Programming Languages," in *Proc. 7th ACM Symposium on Principles of Distributed Computing (PODC)*, pp. 210–222, 1988.

[3] Z. Goertzel and Oruži, "MeTTaIL: Contract-First Boot Compilation for Direct C MeTTa Runtimes," Working paper, 2026.

[4] L. G. Meredith and M. Radestock, "A Reflective Higher-Order Calculus," *Electronic Notes in Theoretical Computer Science*, vol. 141, no. 5, pp. 49–67, 2005.

[5] L. G. Meredith, "Continued Interactive GSLTs and the Cost Endofunctor: Wrapping by Construction," Working paper, 2026.

[6] Mettapedia, "Pure Rho and Cost: One Authored Calculus, Named Extensions, and Corrected Hypotheses," Working paper, 2026.

[7] R. Atkey, "Parameterised Notions of Computation," *Journal of Functional Programming*, vol. 19, no. 3–4, pp. 335–376, 2009.

[8] S. Katsumata, "Parametric Effect Monads and Semantics of Effect Systems," in *Proc. POPL*, 2014.

[9] D. Orchard, P. Wadler, and H. Eades III, "Unifying Graded and Parameterised Monads," in *Proc. MSFP*, 2020.

[10] V. Khudobakhshov, "JeTTa — Performance: Compiling MeTTa to JVM Bytecode," Slide deck, July 2026.

[11] Oruži and Z. Goertzel, "JeTTa: MeTTa Compiled to JVM Bytecode — A Source-Level Review, Cross-Engine Benchmark Study, and Defect Catalog," Working paper, 2026.

[12] K. H. Knuth and J. Skilling, "Foundations of Inference," *Axioms*, vol. 1, no. 1, pp. 38–73, 2012.